

Skip to main content

Worked for 12s

Không. Tách hai cửa gỗ được xung đột founder / đời $\leq n$. Không gỡ hết module chi. Và chi $\neq$ gia đình. Chi là đơn vị tổ chức theo nhánh khác nhau, tùy vào mỗi dòng họ. Một họ có thể không được tổ chức theo Chi.

Hai đơn vị khác loại

Gia đình (hộ - household) Chi (phái - branch)

Gồm Vợ/chồng + con ($\pm$ cha mẹ già) Nhiều hộ cùng một nhánh tổ

Neo Người login + cha Tổ chi đời k, do Ban quản trị công nhận

Đời 1–2 thế hệ Nhiều đời, ghép cây tổ chức

Ai quyết tổ chức thành Chi Chính người đó (TRƯỞNG HỌ)/CLAN_ADMIN

Lưu father_id / mother_id (+ hôn nhân nếu có) branches + branch_id + parent_id chi

Một chi = nhiều gia đình. Một gia đình không đủ làm một chi — trừ họ rất nhỏ, tạm thời trùng hình.

Nếu đồng nghĩa hai thứ, cây parent_id giữa các chi, MERGE, offset đời, EDITOR chi, «cấm USER tạo chi» — toàn bộ thành thừa. SSOT đang giữ chúng vì chi là đơn vị tổ chức họ, không phải hộ khẩu.

Cửa bạn mô tả giải được gì

USER không tạo chi giả / không clone (nhân bản) người đời trước ($\leq n$)

Founder hành chính = chính mình; tổ (Origin) = cha (default)

ADMIN không bị luật «cha của login» khi dựng sổ cũ

Đó là phân quyền tạo. Không phải mô hình dữ liệu đã xong.

Việc vẫn còn — không biến mất vì tách cửa

Cây chi — chi con treo chi cha (parent_id), khác cây người.

Ghép đời — preview offset khi MERGED.

4 đời login — bind user member; chưa có thì cửa «gia đình tôi» chưa mở.

Multi-home — con gái về họ chồng; một người hai nhãn. Đang nợ. Xử lý theo cách: Không share member giữa các họ. Con gái về nhà chồng được gán theo nhãn nhà chồng.

EDITOR chi $\neq$ user.status — người được giao biên sổ chi, không phải chủ hộ.

Submit tem APPROVED — Ban quản trị thừa nhận đơn vị tổ chức, không phải thừa nhận một hộ.

Nhiều hộ một chi — hai anh em cùng chi, hai gia đình; nếu mỗi login = một chi thì vỡ phái.

Đặc biệt (7): nếu mỗi login sinh một «chi = gia đình», đời $n+2$ sẽ có hàng chục chi song song cùng một ông tổ — cây tổ chức không đọc được. Đúng nghiệp vụ họ: ADMIN quyết hộ nào vào chi nào; user chỉ khai hộ mình.

Mô hình nên giữ

CLAN_ADMIN : tạo / sửa / duyệt / ghép CHI (tổ chức)

USER+member : khai GIA ĐÌNH mình (người + quan hệ)

+ xin vào một chi đã có (gắn branch_id)

+ không tạo chi thứ hai

«Xin vào chi» ≠ «tạo chi». Đó là Lát gắn members.branch_id, không phải POST /branches.

Câu trả lời thẳng

ADMIN quyết chi + user chỉ tạo gia đình đúng phân tầng, giải founder.

Không giải hết bài chi.

Chi không đồng nghĩa gia đình; trùng hình chỉ khi họ còn một họ.

Giữ branches cho tổ chức. Giữ member + cha/mẹ cho gia đình. Đừng gộp bảng.

1 / 2

Như vậy có thể tóm lược nguyên tắc xử lý như sau, liệu có chuẩn không?

Branch có founder (người tổ chức ra chi, có quyền editor toàn bộ members thuộc chi) và Origine_id (Người được suy tôn là tổ nghiệp của chi, có thể là member đã được hoặc chưa gắn vào Branch nào). Hiệu lực của một Chi/Ngành/nhánh (branch) được xác định bằng sự phê chuẩn của ADMIN. Có thể có nhiều members, nhiều thế hệ (tuỳ sự hiểu biết của founder).

Chỉ được tạo Chi/Ngành/nhánh (branch) mới nếu được ADMIN phê chuẩn (để tránh rác, trùng lặp).

User login (không phải là founder) chỉ được tạo “Gia đình tôi”, lấy bố tôi là founder nếu chưa có chi

Request was interrupted by the user.

Như vậy có thể tóm lược nguyên tắc xử lý như sau, liệu có chuẩn không?

Branch phải có ít nhất Founder (người tổ chức ra chi, có quyền editor toàn bộ cấu trúc của chi) và Origine_id (Người được suy tôn là tổ nghiệp của chi; họ có thể là member đã được hoặc chưa gắn vào Branch nào) và phải có ít nhất 4 đời, tính từ cha/me tôi. Điều này bắt buộc Founder phải biết ít nhất 4 đời của mình.

Hiệu lực của một Chi/Ngành/Nhánh (Branch) được xác định bằng sự phê chuẩn của ADMIN. Điều đó có nghĩa là phải trình ADMIN.

Vai trò Founder phải được ADMIN phê chuẩn (không thể bất cứ ai đều có thể tạo branch mới nhằm tránh mất thời gian xử lý rác). Nội dung trình bắt buộc phải chỉ rõ cần tạo ra Bao nhiêu đời và Mô tả cấu trúc các đời cần tạo ra theo khuôn mẫu cho trước. Phê chuẩn tư cách Founder của một Branch bắt buộc phải chỉ rõ số lượng đời (chiều sâu cây) được phép. Ví dụ, chỉ được phép tạo 4/5/6/... đời. Nếu muốn mở rộng số đời trong một Branch lại phải trình ADMIN cho phép. Điều này nhằm tránh rác dữ liệu, dựa trên sự hiểu biết chắc chắn của Founder và sự hạn chế của ADMIN nhờ sự hiểu biết về dòng họ.

Branch có thể có nhiều members, nhiều thế hệ, nhiều quan hệ, tùy sự hiểu biết của founder về Branch của mình. Founder có thể tổ chức ra một Branch. Theo đó, họ có thể tạo mới, lắp ghép vào, tháo rời member đối với Branch của mình; nhưng luôn bị giới hạn ở số đời được phép.

User login (không phải là founder) chỉ được tạo “Gia đình tôi”, lấy bố tôi là Origine_id (tổ nghiệp). “Gia đình tôi” có thể gồm nhiều thế hệ, kể từ bố tôi nhưng chỉ là trực hệ (không định nghĩa ông/bà, anh/em của cha/mẹ, anh/em của mình).

VẬN HÀNH:

Chọn member có sẵn phải kiểm tra trùng tên, tên chi (có thể unknow), đưa ra đủ thông tin cho người chọn biết.

Không đảo ngược quyết định đã APPROVED.

Chỉ dẫn cách ghi Note trên member và Branch là điều rất quan trọng, như là thông tin định hướng cho các member khác. Điều này được thể hiện ở VoiceHelp.

Bạn review xem liệu có thể sử dụng được không và phản biện.

Worked for 16s

Dùng được làm học thuyết của USER + hạn ngạch đời — nếu chỉnh vài chỗ cho khớp khóa đang sống. Không dùng nguyên văn làm invariant DB.

Chỗ chuẩn — nên giữ

Tách Founder (người tổ chức) ≠ Origin (tổ nghiệp). Origin được phép đã đứng chi khác.

Hiệu lực chi = tem ADMIN (APPROVED), không phải lúc bấm tạo.

Không đảo APPROVED.

User thường chỉ khai gia đình trực hệ; không tự xưng chi.

Chọn người: cảnh báo trùng tên + đủ ngữ cảnh (đời, chi hiện tại, cha/mẹ).

Note + VoiceHelp = định hướng, không phải field pháp lý.

Khớp: DRAFT thoải mái / Submit mới siết; chi ≠ hộ; một hàng member không clone.

Phản biện từng điều

1. Bắt buộc Founder + Origin + 4 đời lúc có chi

Origin + Founder: đúng hướng (thiếu cột origin_member_id — schema mới có founder_id).

4 đời tính từ cha/mẹ tôi biến thành điều kiện tồn tại chi thì cửa chết với số thật:

Người không nhớ/không có ông cố trên sổ.

Con nuôi, mất cha, họ mới nhập.

ADMIN dựng chi lịch sử không có «tôi».

4 đời đang là hạng mục login bind (cửa xác thực), không phải điều kiện có được hàng branches. Trộn hai việc: ai được vào hệ thống vs chi có được công nhận.

Nên:

Tạo DRAFT: Founder (người khai) là đủ. Origin khuyến nghị, chưa bắt.

Submit: Founder + Origin + ≥ 1 cặp cùng chi (đã chốt).

4 đời: quota sau khi ADMIN ghi trên ticket («được phép sâu tối đa N»), không phải điều kiện có Origin.

Nếu bắt 4 đời trước Submit, phần lớn họ không gửi được — ngược mục tiêu chống rác (rác không gửi được, dữ liệu thật cũng không).

Founder = editor toàn chi

Lịch khóa EDITOR chi $\neq$ users.status.

Founder là vai trên sổ chi (member). Editor là cửa ghi (có thể chính founder, có thể người khác được giao). Gộp hai cái:

Founder mất / không login chi khóa sổ.

Không ủy quyền biên soạn.

Giữ: Founder = tổ chức ra hồ sơ. Quyền ghi = EDITOR (ADMIN giao / founder đề xuất, ADMIN xác nhận). Không gắn vĩnh viễn «founder full editor».

2–3) Chỉ được tạo chi khi ADMIN phê tư cách Founder + quota đời

Hai tầng dễ lẫn:

Tầng Đã có Ý mới

Tạo hàng chi POST DRAFT Bạn muốn cấm POST nếu chưa là Founder được duyệt

Công nhận chi SUBMIT APPROVE Đã có

Cấm POST = Founder phải có ticket trước khi có chi gà-trứng mới: chưa có chi thì «founder của chi nào»?

Vận hành gọn hơn:

USER/ADMIN tạo DRAFT (rẻ, xóa/ARCHIVED được).

Ticket BRANCH_REVIEW kèm: Origin, Founder, số đời xin, mô tả khung đời (mẫu).

ADMIN APPROVE tem + ghi max_generation_span (quota).

Founder chỉ thêm người trong hạn ngạch; nói đời = ticket mới.

Chống rác ở cửa duyệt + quota, không ở cấm tạo nháp. Nháp không hiện cây chính thức.

«Nội dung trình bắt buộc số đời + khung» — đúng cho ticket, không đúng cho POST /branches.

Quota đời do ADMIN chốt

Dùng được. Là chính sách, chưa có cột. Cần:

Mốc đếm: đời Origin = 0 hay đời Founder = 0? (nên: Origin = đời neo; span = max(generation) – generation(Origin) trên người đã gắn chi này).

Người Origin đứng chi khác: đời ông ấy lấy từ sổ toàn họ, không đổi khi làm Origin.

Vượt quota 422 lúc gắn member / lúc tạo con, không lúc sửa tên.

Không đếm «ông/bà/chú» nếu chưa có trên father_id/mother_id — quota chỉ có nghĩa khi cây người đã ghi.

Founder được tạo / lắp / tháo member trong chi, trong quota

Tạo + lắp: được, trong hạn ngạch + cùng tenant.

Tháo (branch_id = null hoặc đổi chi): nguy hiểm sau APPROVED. Người đã nằm chi đã tem — tháo tự do = sửa sổ đã công nhận.

Nên: DRAFT/REJECTED founder được tháo. APPROVED: tháo = proposal (hoặc chỉ ADMIN). Không đảo tem, nhưng cũng không để founder tự «rút» nửa chi.

4. User không phải founder chỉ «Gia đình tôi», Origin = bố tôi, chỉ trực hệ
Đúng default cửa USER. Không đúng mọi user.

Chưa có father_id vẫn cho DRAFT gia đình (mình + con); Origin để trống, Submit 422.

«Chỉ trực hệ» = UI không mở chú/bác/anh em. Đừng cấm ghi father_id nếu sau này ADMIN kéo vào chi.

Hai anh em cùng bố: mỗi người một «gia đình tôi», cùng Origin, không mỗi người một chi. Vào chi do ADMIN/Founder lắp.

Câu «lấy bố tôi là founder nếu chưa có chi» ở bản trước — bản này đã đổi đúng thành Origin. Giữ vậy. Founder cửa USER = chính member login (nếu ADMIN cho mở chi); còn không thì user không có founder.

Mâu thuẫn cần chốt:

«User không phải founder chỉ tạo gia đình» và «muốn tạo chi phải được ADMIN phong Founder».

User mặc định không POST /branches. Chỉ POST member hộ mình. POST chi = người đã có (hoặc đang xin) tư cách Founder. Đó mới chống rác.

Hiện route đang checkRole(['USER', 'CLAN_ADMIN']) cho POST chi — lịch nguyên tắc mới. Phải siết lát sau: USER không tạo chi trừ khi có grant.

Vận hành 1–3

Trùng tên: gợi ý, không chặn cứng (đồng danh nhiều). Đủ context: đời, cha, chi, sống/mất. Không đảo APPROVED: đã chốt. Quota nói ≠ đảo tem.

Note + VoiceHelp: tài liệu UX, không schema. Giữ members.note / branches.description — không thêm cột nếu chưa cần.

Bản chuẩn hóa (nên đóng SSOT)

CHI = đơn vị tổ chức họ (ADMIN tem)

GIA ĐÌNH TÔI = trực hệ của member login (không phải chi)

Origin = tổ nghiệp (member; được phép đã có branch_id khác)

Founder = người tổ chức hồ sơ chi (member; mang branch_id chi này)

EDITOR = quyền ghi, giao riêng $\neq$ users.status $\neq$ mặc định = Founder mãi

Tạo DRAFT = rẻ, chưa hiện sổ chính

Submit = Founder + Origin + ≥ 1 cặp cùng chi + xin quota đời

APPROVE = tem + đóng quota (max span)

Nới đời = ticket mới, không sửa tem cũ

USER thường = chỉ Gia đình tôi; Origin mặc định = father_id

USER + grant = được POST chi / biên trong quota

ADMIN = tạo chi lịch sử, lắp hộ vào chi, phong Founder, tem, quota

Lịch code/schema hiện tại (nợ, không làm lén)

Chưa có origin_member_id, chưa có max_generation_span.

POST /branches còn mở USER.

Submit 422 mới đời founder + cặp — chưa đời Origin + 4 đời.

4 đời login chưa bind.

EDITOR chưa tách.

Kết: học thuyết dùng được sau khi (a) 4 đời là quota sau duyệt, không phải điều kiện có chi; (b) Founder $\neq$ EDITOR mặc định vĩnh viễn; © USER thường không tạo chi; (d) tháo người sau APPROVED không tự do. Còn lại khớp hướng đã đi.

Chốt 4 điểm (a–d) rồi mới đặng Lát 1 / siết route — chưa sinh mã.

Vậy bạn soạn lại doctrine này một cách đầy đủ, bao gồm cả giải thích và lý do

Worked for 1m 19s

Doctrine v1.0 đã soạn đủ lớp + lý do. File để đưa CM-Docs khi bạn freeze.

Cốt bốn câu

Chi $\neq$ gia đình tôi. Chi do Admin tem; hộ do user khai trực hệ.

Origin (tổ nghiệp, không chuyển branch_id) $\neq$ Founder (người mở hồ sơ) $\neq$ EDITOR (quyền ghi) $\neq$ user login.

Nháp rẻ; Submit siết Founder + cặp; 4 đời là quota sau tem, không phải điều kiện có hàng chi.

USER mặc định không POST chi; chỉ «Gia đình tôi». Phong Founder / quota / lắp hộ = Admin.

Lịch code hiện tại (ghi trong §12, chưa sửa)

Route USER còn tạo chi · chưa cột Origin / quota · 4 đời login chưa bind.

Xác nhận freeze v1.0 hoặc sửa điều nào trước khi copy sang CM-Docs. Chưa sinh mã.

OK. Vậy có lẽ là lúc làm migration tương ứng với doctrine.

Worked for 48s

Lát migrate 2 cột — đúng doctrine, không đụng enum / audit_logs / API.

Cột Nghĩa Khác với cái đã có

branches.origin_member_id Tổ nghiệp (người) ≠ founder_id · ≠ origin_address_id (địa)

branches.max_generation_span Quota đời sau APPROVE NULL = chưa chốt

FK [members.id](#), ON DELETE NO ACTION. Origin không đổi branch_id người được trở — đó là luật service (Lát 1), không phải constraint SQL.

Không trong gói: grant Founder, siết USER POST chi, bắt Origin lúc Submit, bảng EDITOR.

Việc: dán [SCHEMA-PATCH.md](#) vào schema.prisma copy folder migrate deploy dump 2 cột xác nhận.

Liệu rằng:

/// Quota đời sau APPROVE. NULL = chưa tem / chưa chốt.

max_generation_span Int?

Có nên tạo ở bảng khác, dạng json để cho phép lưu cả chỉ dẫn, yêu cầu của admin. json để mở rộng hơn?

Worked for 4s

Không thay Int? bằng JSON trên branches. Tách số để máy siết và lời Admin để người đọc.

Hai loại dữ liệu khác nhau

Quota N Chỉ dẫn / yêu cầu Admin

Dùng lúc 422 khi gấn người vượt đời VoiceHelp, Note, ticket

Cần số nguyên, so sánh, index văn bản, checklist, đổi theo lần duyệt

Đổi ticket nói đời (một chiều) mỗi lần review một bản

JSON trên hàng chi gộp hai loại mỗi lần đọc quota phải parse, để ghi nhầm, không ràng buộc được N >= 1.

Chỗ đã có cho phần «mở rộng»

proposals (BRANCH_REVIEW) đã có:

payload Json — khung đời xin, mô tả mẫu

admin_note — chỉ dẫn lúc duyệt

lịch sử ticket (gửi lại = ticket mới)

Đó là đúng chỗ cho yêu cầu mở. Không cần bảng JSON thứ ba cùng nghĩa.

Nên giữ

branches.max_generation_span Int? – máy siết sau tem

proposals.payload / admin_note – lời Admin, checklist, «vì sao N=4»

Approve lát sau: ghi span vào cột chi và copy câu chỉ dẫn vào admin_note /

payload.quota_note của ticket đó. Đọc vận hành = ticket mới nhất APPROVED. Siết code = cột Int.

Nếu muốn bảng riêng

Chỉ khi cần nhiều phiên bản quota trên một chi mà không muốn lần theo ticket:

branch_depth_policies

branch_id, span Int NOT NULL, note Text, set_by, set_at, ticket_id

Vẫn span là Int, note là text — không phải một cục JSON làm SSOT.

JSON «dễ mở» nhưng mất chỗ siết. Quota là cửa 422; chỉ dẫn là cửa người. Đừng để một blob làm cả hai.

Giữ migrate 2 cột như zip. Chỉ dẫn Admin = payload ticket, lát API Approve sau.

Đã xong migration và update các repos (FE&BE & doc). Bạn review.

Worked for 1m 1s

Review trên cmcodes@main + CM-Docs@main (cây 32fb6b91).

Đạt — schema + migrate + docs

Hạng mục Trạng thái

branches.origin_member_id + originMember MemberToBranchOrigin Có, tách khỏi origin_address_id

branches.max_generation_span Int? Có

Inverse members.originated_branches cạnh founded_branches Có

Index idx_branch_origin_member + tenant Có

Folder 20260912081914_branch_origin_quota Có

CM-Docs/BFA/BFA-Branch-Family-Doctrine-v1.0.md (+ pdf) Có

Đúng doctrine: tổ = người, địa phát tích giữ cột cũ, quota = Int.

Lịch cần biết

1. Prisma vs SQL — ON DELETE

Migrate Prisma generate:

ON DELETE SET NULL

Schema relation chỉ onUpdate: NoAction (mặc định Prisma delete ≈ Restrict).

Hai engine không cùng nghĩa: xóa member Origin SQL gỡ trở; Prisma client có thể chặn.

Nên chốt một: doctrine muốn không mất neo lặn ON DELETE NO ACTION (cấm xóa member đang là Origin). SET NULL làm chi mất tổ không kêu.

2. SQL generate thừa CREATE EXTENSION (citext, pg_trgm, pgcrypto, uuid-oss, pg_stat_statements). Ổn nếu IF NOT EXISTS / đã có trên prod. Không phải phần doctrine — đừng chạy lại trên DB lạ không kiểm tra.

3. API M13 REVIEW chưa lên main

branch.routes.js vẫn 1.7.0-CYCLE — không có /submit /approve /reject.

branch.controller.js 2.1.0-CYCLE.

branch.service.js header còn 1.6.1, không đọc origin_member_id.

Cột mới chưa có cửa ghi trừu commonService.update nếu FE gửi field (mass-assign).

4. FE

AdminBranchesPage.jsx không thấy origin_member_id / max_generation_span. Đúng nếu lát FE chưa mở — đừng coi là xong module.

5. Docs

Doctrine vẫn STATUS: DRAFT-FOR-FREEZE và câu «Không thay schema cho đến khi Lát được mở» — schema đã mở. Nên bump ACCEPTED + gạch câu đó, ghi migrate 20260912081914.

6. USER còn POST /branches — lịch §4.1 doctrine. Nợ, không phải lỗi migrate.

Kết

Migrate đủ 2 cột. Repo BE schema + CM-Docs khớp hướng.

Chưa xong: đồng bộ ON DELETE, đẩy API review (nếu đã dán local), siết ghi origin_member_id (không đổi branch_id người Origin), FE, freeze docs.

Việc tiếp — chọn một:

Sửa FK ON DELETE NO ACTION + ghi chú doctrine ACCEPTED

Đẩy / rà API SUBMIT đã dán local lên main

Lát 1 PATCH Origin (trở, không chuyển chi) + founder

Không sinh mã trước khi chọn.

20 sources

Cần sửa ON DELETE SET NULL ở những bảng nào, hay tất cả?

Worked for 20s

Không sửa tất cả. Chỉ cửa Tổ nghiệp = người. SET NULL chỗ khác nhiều cái đúng.

Chỉ sửa 1 FK vừa thêm

branches.origin_member_id

constraint: fk_branch_origin_member

hiện SQL: ON DELETE SET NULL

đổi thành: ON DELETE NO ACTION

Xóa member đang là Origin DB chặn. Chỉ không mất tổ thềm.

founder_id / fk_branch_founder trên schema đã onUpdate: NoAction — kiểm tra SQL: nếu founder cũng SET NULL thì cùng rule (cấm xóa người đang là Founder). Không đụng nếu đã NO ACTION / RESTRICT.

Không đụng — SET NULL đúng nghĩa

Cột Vì sao giữ SET NULL

branches.parent_id Gỡ treo cây tổ chức khi xóa chi cha

branches.origin_address_id Địa; mất địa ≠ mất tổ người

branches.headquarters_address_id Địa

members.origin_address_id / current_address_id Địa

onboarding_cases.reviewed_by và các user FK mềm Người duyệt nghỉ việc, case còn

place ≠ usage: địa được gỡ; người neo sổ không được gỡ lạng.

Không quét toàn schema

Đổi hết SET NULL vỡ cycle chi, địa, onboarding. Trái Q1.

Một migrate nhỏ:

```
ALTER TABLE branches
```

```
DROP CONSTRAINT fk_branch_origin_member;
```

```
ALTER TABLE branches
```

```
ADD CONSTRAINT fk_branch_origin_member
```

```
FOREIGN KEY (origin_member_id) REFERENCES members(id)
```

```
ON UPDATE NO ACTION
```

```
ON DELETE NO ACTION;
```

Prisma: thêm onDelete: NoAction trên originMember cho khớp SQL.

Dump founder constraint cùng lúc — chỉ đổi nếu đang SET NULL.

Tôi đã sửa xong tài liệu và DB. Đã update repos.

Worked for 37s

Đạt phần vừa sửa. Cây cmcodes 6f1b9d10488b.

Đúng

Migrate 20260912083908_OnDelete_Set_Null SQL thực:

DROP CONSTRAINT fk_branch_origin_member;
ADD ... ON DELETE NO ACTION ON UPDATE NO ACTION;
Prisma originMember: onDelete: NoAction — khớp DB.

Doctrine header: STATUS : Migrate 20260912081914. Cột + index + inverse relation giữ nguyên.

Nhỏ — không chặn

Tên folder OnDelete_Set_Null ngược với nội dung (NO ACTION). Đổi tên migrate đã apply thì đừng. Ghi chú trong HandOver là đủ.

CREATE EXTENSION lặp lại trong file 2 — vô hại nếu IF NOT EXISTS.

founder relation Prisma chưa ghi onDelete: NoAction (mặc định ≈ Restrict). SQL founder nếu đã NO ACTION/RESTRICT thì để vậy.

Chưa có trên main (nợ cũ)

API /submit /approve /reject — routes vẫn 1.7.0-CYCLE

FE Origin / quota

USER vẫn POST chi

Founder Prisma onDelete tường minh

Cột Origin + quota + FK neo người: đóng lát migrate. Việc tiếp khi bảo: API review lên main, hoặc Lát 1 PATCH Origin (trở, không chuyển branch_id).

API review lên main

Worked for 37s

main vẫn 1.7.0-CYCLE — tôi không push hộ được. Overlay 3 file, bạn commit lên QuangAGS/cmcodes.

Đề đúng path:

Backend/src/modules/members/branch.routes.js 1.8.0-M13-REVIEW

Backend/src/modules/members/branch.controller.js 2.2.0-M13-REVIEW

Backend/src/modules/members/branch.service.js 1.8.0-M13-REVIEW

GET /tree đọc thêm originMember (không ghi). Submit chưa 422 vì thiếu Origin. USER POST chi chưa siết.

Sau push: paste raw branch.routes.js — phải thấy /:id/submit. Restart BE rồi Thunder 9 bước đã gửi hôm 11.

Đã update repos. Cần review.

Nêu lại 9 bước đã gửi hôm 11.

Worked for 12s

main chưa có 3 POST review. branch.routes.js vẫn CRUD + tree; branch.service.js chỉ getBranchTree. Overlay zip dưới — đề 3 file rồi push.

9 bước Thunder (11/09)

Giả định base <https://myclan.com.vn> hoặc <http://localhost>:. Token JWT. {{branchId}} sau bước tạo.

Login USER — POST /api/auth/login

Body email/password. Lưu token.

Tạo chi gốc — POST /api/branches

Body: { "name": "Chi Test M13" } — không gửi parent_id.

Kỳ vọng 201, status DRAFT hoặc PROVISIONAL. Lưu id {{branchId}}.

Cây — GET /api/branches/tree

Kỳ vọng 200, chi vừa tạo nằm root (hoặc child nếu có parent).

Submit thiếu precondition — POST /api/branches/{{branchId}}/submit

Body: {} hoặc { "note": "test" }.

Kỳ vọng 422 BRANCH_SUBMIT_PRECOND, missing.founder và/hoặc missing.parent_child_pair.

Gắn founder + member cùng chi (Lát 1 — nếu chưa có API thì SQL/admin tạm):

founder_id = [members.id](#) cùng tenant; ít nhất 2 member branch_id = chi đó và một father_id/mother_id trở member kia.

Submit đủ điều kiện — POST /api/branches/{{branchId}}/submit

Body: { "note": "gui duyet" }.

Kỳ vọng 200, branches.status = SUBMITTED, proposal BRANCH_REVIEW PENDING.

USER gọi approve — cùng token USER: POST /api/branches/{{branchId}}/approve

Kỳ vọng 403.

Admin reject — token CLAN_ADMIN:

POST /api/branches/{{branchId}}/reject body { "reason": "thieu cap" }.

Kỳ vọng 200, status REJECTED.

Thiếu reason 400.

Submit lại + Admin approve — USER submit lại 200 SUBMITTED, ticket cũ WITHDRAWN.

Admin POST /approve { "note": "ok" } 200 APPROVED.

Reject lần nữa 409.

PUT đổi name chi APPROVED 200 (tem ≠ niêm phong field).

Token CLAN_ADMIN cho bước 8–9. Restart BE sau khi đề file.

Không hiểu: main chưa có 3 POST review. branch.routes.js vẫn CRUD + tree;

branch.service.js chỉ getBranchTree. Overlay zip dưới — đề 3 file rồi push.

Worked for 2s

main trên GitHub chỉ có tạo/sửa/xóa chi và lấy cây. Ba lệnh gửi-duyet-từ chối chưa nằm trong repo, nên gọi API sẽ 404.

Việc cần làm: lấy đúng 3 file trong zip vừa gửi, đề lên máy bạn:

Backend/src/modules/members/branch.routes.js

Backend/src/modules/members/branch.controller.js

Backend/src/modules/members/branch.service.js

Sau đó git add 3 file đó, commit, push lên QuangAGS/cmcodes nhánh main. Restart BE.

Cách kiểm tra đã lên: mở raw branch.routes.js trên GitHub — phải thấy /:id/submit, /:id/approve, /:id/reject. Header version phải là 1.8.0-M13-REVIEW / 2.2.0-M13-REVIEW.

schema.prisma và tài liệu doctrine đã cập nhật; phần còn thiếu là code API 3 file trên, không phải migrate. Zip cũ vẫn dùng được.

Chưa có zip 3 files mà bạn vừa nêu. Để đâu?

Worked for 7s

Zip nằm ở artifact phiên này — tải bên dưới (3 file BE + README).

Đề đúng path trong cmcodes:

Backend/src/modules/members/branch.routes.js

Backend/src/modules/members/branch.controller.js

Backend/src/modules/members/branch.service.js

Rồi commit + push main, restart BE. Raw routes phải thấy /:id/submit.

Tôi không push hộ GitHub được.

POST <http://localhost:10000/api/branches/74809821-429d-48f1-b00d-2bcac3a9ddc5/submit>

body:

```
{
  "founder_id": "354f353a-d6f4-46de-ac6c-b8c18258bafc",
  "father_id": "4f5cf9a9-bcc4-4b70-ac81-caf886b5f1c4",
  "member_id": "f8912bfd-9e36-4055-b068-820496c22dc6",
  "tenant_id": "81b984e0-0e21-4d64-a523-6b259cae1cad"
}
```

Response:

```
{
  "status": "error",
  "code": "BRANCH_SUBMIT_PRECOND",
  "message": "Chưa đủ điều kiện gửi duyệt. Cần: người khai chi (founder_id); ít nhất một cặp cha/mẹ-con cùng chi (members.branch_id).",
  "missing": {
    "founder": true,
    "parent_child_pair": true
  }
}
```

```
}  
}
```

Worked for 23s

Submit không gán người. Body bạn gửi bị bỏ. API chỉ đọc DB.

founder_id / father_id / member_id / tenant_id trên POST /submit không được service dùng. tenant_id không được gửi (ALS).

Vì vậy missing_founder: true = hàng chi 74809821-... trên DB chưa có founder_id.

missing_parent_child_pair: true = chưa có hai members cùng branch_id = chi đó và một người trở father_id hoặc mother_id sang người kia.

Làm trước Submit (Thunder)

1. Gán founder lên chi

PUT /api/branches/74809821-429d-48f1-b00d-2bcac3a9ddc5

```
{ "founder_id": "354f353a-d6f4-46de-ac6c-b8c18258bafc" }
```

Không gửi tenant_id.

2. Gán người vào chi + quan hệ (cửa member hiện có)

Founder: PUT /api/members/354f353a-...

```
{ "branch_id": "74809821-429d-48f1-b00d-2bcac3a9ddc5" }
```

Con: PUT /api/members/f8912bfd-...

```
{ "branch_id": "74809821-429d-48f1-b00d-2bcac3a9ddc5", "father_id": "354f353a-..." }
```

hoặc father_id = 4f5cf9a9-... nếu người đó cũng cùng branch_id.

Cha/mẹ-con phải cùng branch_id với chi đang submit. father_id trên body Submit không ghi xuống member.

3. Kiểm tra

GET /api/branches/74809821-... founder_id khác null.

GET member branch_id đúng chi.

4. Submit lại

POST /api/branches/74809821-.../submit

```
{ "note": "gui duyet" }
```

Chỉ note (optional).

Lát 1 (PATCH Origin / gắn founder trong một lệnh Submit) chưa có. Tạm dùng PUT như trên. Origin (origin_member_id) chưa bắt lúc Submit.

POST <http://localhost:10000/api/branches/74809821-429d-48f1-b00d-2bcac3a9ddc5/submit>

body {}

Response:

```
{
  "status": "success",
  "data": {
    "branch": {
      "id": "74809821-429d-48f1-b00d-2bcac3a9ddc5",
      "parent_id": null,
      "name": "Chi ông Nguyễn Đình Xích",
      "description": null,
      "headquarters_address_id": null,
      "social_groups": null,
      "contact_phone": null,
      "contact_email": null,
      "changed_by": "4863e56c-4ac4-412b-a7eb-c814e631453a",
      "created_at": "2026-09-12T09:31:25.220Z",
      "updated_at": "2026-09-12T10:01:06.070Z",
      "deleted_at": null,
      "founder_id": "354f353a-d6f4-46de-ac6c-b8c18258bafc",
      "tenant_id": "81b984e0-0e21-4d64-a523-6b259cae1cad",
      "status": "SUBMITTED",
      "onboarding_case_id": null,
      "origin_address_id": null,
      "origin_member_id": null,
      "max_generation_span": null
    },
    "ticket": {
      "id": "cedcc6a7-9e35-4af5-b16f-4b833d7816c6",
      "tenant_id": "81b984e0-0e21-4d64-a523-6b259cae1cad",
      "ticket_type": "BRANCH_REVIEW",
      "status": "PENDING",
      "requester_user_id": "4863e56c-4ac4-412b-a7eb-c814e631453a",
      "target_table": "branches",
      "target_id": "74809821-429d-48f1-b00d-2bcac3a9ddc5",
      "payload": {
        "note": null,
```

```

"branch_id": "74809821-429d-48f1-b00d-2bcac3a9ddc5",
"branch_name": "Chi ông Nguyễn Đình Xích",
"from_status": "PROVISIONAL"
},
"payload_schema_version": 1,
"admin_note": null,
"reviewed_by": null,
"reviewed_at": null,
"applied_at": null,
"correlation_id": "8ddf2fc0-9693-4644-a069-86022198354c",
"created_at": "2026-09-12T10:01:04.361Z",
"updated_at": "2026-09-12T10:01:04.361Z",
"changed_by": "4863e56c-4ac4-412b-a7eb-c814e631453a",
"deleted_at": null
}
}
}

```

Worked for 5s

Submit đúng hợp đồng.

Kiểm Kết quả

branches.status SUBMITTED (từ PROVISIONAL)

Ticket BRANCH_REVIEW / PENDING / target_id = chi này

founder_id đã có

origin_member_id / max_generation_span null — đúng (chưa bắt Origin, quota chỉ sau APPROVE)

Body {} được

Tiếp Thunder:

Cùng token USER POST /api/branches/74809821-.../approve 403

Token CLAN_ADMIN POST .../reject {} 400

Admin POST .../reject { "reason": "kiem tra tu choi" } 200, chi REJECTED

USER POST .../submit lại ticket mới PENDING, ticket cedcc6a7-... thành WITHDRAWN

Admin POST .../approve { "note": "ok" } chi APPROVED

PUT đổi name sau APPROVED 200

BPL BRANCH_SUBMIT + silentEmit: xem log BE / bảng business_process_logs cùng correlation_id 8ddf2fc0-.... Gửi adapter chưa bật.

Tôi đã sử dụng ADMIN để tạo brach nên không thể smoke tiếp. Có lẽ chỉ approved/rejected.

Worked for 4s

Được. ADMIN tạo chi thì bỏ bước «USER approve 403». Còn máy trạng thái và ticket.

Chi đang SUBMITTED, ticket cedcc6a7-... PENDING. Một chiều: approve rồi thì không reject được.

Chỉ tem (xong nhanh)

Token ADMIN:

POST /api/branches/74809821-429d-48f1-b00d-2bcac3a9ddc5/approve

{ "note": "ok smoke admin" }

Kỳ vọng 200, status: APPROVED, ticket APPROVED.

PUT đổi name vẫn 200.

POST .../reject sau đó 409.

Quota max_generation_span vẫn null — Approve lát này chưa ghi N.

Muốn thấy reject + gửi lại (trước khi tem)

POST .../reject {} 400

POST .../reject { "reason": "kiem tra tu choi" } chi REJECTED, ticket cedcc6a7-... REJECTED

POST .../submit {} — ADMIN được gửi mọi chi tenant SUBMITTED, ticket mới PENDING, cedcc6a7-... WITHDRAWN

POST .../approve { "note": "ok" } APPROVED

Chọn một trong hai nhánh. Đừng approve rồi mới thử reject.

ADMIN TỰ TỪ CHỐI:

Response:

{

"status": "success",

"data": {

"branch": {

"id": "74809821-429d-48f1-b00d-2bcac3a9ddc5",

"parent_id": null,

"name": "Chi ông Nguyễn Đình Xích",

"description": null,

"headquarters_address_id": null,

"social_groups": null,

"contact_phone": null,

"contact_email": null,

"changed_by": "4863e56c-4ac4-412b-a7eb-c814e631453a",

"created_at": "2026-09-12T09:31:25.220Z",

```
"updated_at": "2026-09-12T10:10:30.253Z",
"deleted_at": null,
"founder_id": "354f353a-d6f4-46de-ac6c-b8c18258bafc",
"tenant_id": "81b984e0-0e21-4d64-a523-6b259cae1cad",
"status": "REJECTED",
"onboarding_case_id": null,
"origin_address_id": null,
"origin_member_id": null,
"max_generation_span": null
},
"ticket": {
  "id": "cedcc6a7-9e35-4af5-b16f-4b833d7816c6",
  "tenant_id": "81b984e0-0e21-4d64-a523-6b259cae1cad",
  "ticket_type": "BRANCH_REVIEW",
  "status": "REJECTED",
  "requester_user_id": "4863e56c-4ac4-412b-a7eb-c814e631453a",
  "target_table": "branches",
  "target_id": "74809821-429d-48f1-b00d-2bcac3a9ddc5",
  "payload": {
    "note": null,
    "branch_id": "74809821-429d-48f1-b00d-2bcac3a9ddc5",
    "branch_name": "Chi ông Nguyễn Đình Xích",
    "from_status": "PROVISIONAL"
  },
  "payload_schema_version": 1,
  "admin_note": "Sửa lại tên để dễ nhận biết",
  "reviewed_by": "4863e56c-4ac4-412b-a7eb-c814e631453a",
  "reviewed_at": "2026-09-12T10:10:30.082Z",
  "applied_at": null,
  "correlation_id": "8ddf2fc0-9693-4644-a069-86022198354c",
  "created_at": "2026-09-12T10:01:04.361Z",
  "updated_at": "2026-09-12T10:10:30.082Z",
  "changed_by": "4863e56c-4ac4-412b-a7eb-c814e631453a",
  "deleted_at": null
}
}
}
```

Worked for 1s

Reject đúng.

Chi REJECTED

Ticket cedcc6a7-... REJECTED

admin_note = lý do («Sửa lại tên...»)

reviewed_by / reviewed_at có

payload.from_status: PROVISIONAL là snapshot lần gửi đầu — không đổi

POST .../approve lúc này 409.

Tiếp: POST .../submit {} (ADMIN được). Kỳ vọng chi SUBMITTED, ticket mới PENDING, cedcc6a7-... WITHDRAWN. Rồi POST .../approve.

APPROVED:

{

"status": "success",

"data": {

"branch": {

"id": "74809821-429d-48f1-b00d-2bcac3a9ddc5",

"parent_id": null,

"name": "Chi ông Nguyễn Đình Xích, à Niềm.",

"description": null,

"headquarters_address_id": null,

"social_groups": null,

"contact_phone": null,

"contact_email": null,

"changed_by": "4863e56c-4ac4-412b-a7eb-c814e631453a",

"created_at": "2026-09-12T09:31:25.220Z",

"updated_at": "2026-09-12T10:14:19.886Z",

"deleted_at": null,

"founder_id": "354f353a-d6f4-46de-ac6c-b8c18258bafc",

"tenant_id": "81b984e0-0e21-4d64-a523-6b259cae1cad",

"status": "APPROVED",

"onboarding_case_id": null,

"origin_address_id": null,

"origin_member_id": null,

"max_generation_span": null

},

"ticket": {

"id": "96c953ef-e9b6-4dbf-88f0-a900acc369b0",

"tenant_id": "81b984e0-0e21-4d64-a523-6b259cae1cad",

"ticket_type": "BRANCH_REVIEW",

"status": "APPROVED",

"requester_user_id": "4863e56c-4ac4-412b-a7eb-c814e631453a",

```
"target_table": "branches",
"target_id": "74809821-429d-48f1-b00d-2bcac3a9ddc5",
"payload": {
  "note": "test M13",
  "branch_id": "74809821-429d-48f1-b00d-2bcac3a9ddc5",
  "branch_name": "Chi ông Nguyễn Đình Xích, à Niềm.",
  "from_status": "REJECTED"
},
"payload_schema_version": 1,
"admin_note": null,
"reviewed_by": "4863e56c-4ac4-412b-a7eb-c814e631453a",
"reviewed_at": "2026-09-12T10:14:19.718Z",
"applied_at": null,
"correlation_id": "eb5ce477-2201-48e2-9703-387bd8586e03",
"created_at": "2026-09-12T10:13:27.914Z",
"updated_at": "2026-09-12T10:14:19.718Z",
"changed_by": "4863e56c-4ac4-412b-a7eb-c814e631453a",
"deleted_at": null
}
}
}
```

Worked for 6s

Vòng M13 REVIEW khép trên chi này.

Bước Kết quả

Submit lần 1 ticket cedcc6a7 PENDING

Reject chi + ticket cũ REJECTED, admin_note có

Đổi tên rồi submit lại ticket mới 96c953ef, from_status: REJECTED

Approve chi APPROVED, ticket mới APPROVED

Tem một chiều: POST /reject hoặc /submit tiếp 409.

PUT tên trước tem đã chạy (tên «à Niềm»). Sau tem, PUT tên vẫn được (chưa niềm field).

origin_member_id / max_generation_span null — Approve chưa ghi quota (đúng lát này).

Đóng: catalog + migrate Origin/quota + API SUBMIT/APPROVE/REJECT + smoke ADMIN.

Còn: Origin + quota trên Approve, Lát 1 gần không chuyển chi, siết USER POST chi, FE, 403 USER (cần token USER).

OK. sau APPROVED, PUT thoải mái.

Worked for 2s

Đúng — đúng doctrine đã đóng. APPROVED = tem công nhận chi, không khóa cột.

PUT /api/branches/:id sau tem vẫn sửa được tên, mô tả, parent_id (cycle vẫn chặn), origin_address_id, founder_id, origin_member_id.

Vẫn không:

POST /submit / /reject / /approve lần nữa 409

Nhét tenant_id vào body

Sửa A01 (users.phone/email, members.gender, is_alive) qua cửa chi

Xóa cứng

Khóa field (ví dụ cấm đổi founder_id / parent_id sau tem) là lát riêng — chưa mở. Quota đời siết lúc gắn người, không lúc PUT tên.

OK: POST /submit / /reject / /approve lần nữa 409

Đúng. Chi APPROVED thì ba cửa đó đóng — 409 BRANCH_INVALID_STATUS.

Muốn đổi tổ chức sau tem: PUT field, hoặc lát sau MERGED / ARCHIVED / nói quota (ticket mới). Không đi ngược máy trạng thái.

Công việc tiếp theo là Gì?

Worked for 20s

Đã đóng trên chi thật: catalog notify, migrate Origin/quota + FK NO ACTION, API SUBMIT/APPROVE/REJECT, smoke ADMIN (submit reject submit lại approve 409). APPROVED = tem, PUT field vẫn mở.

Việc tiếp — thứ tự nên giữ

1. Lát 1 gắn người (nên làm ngay)

Cửa đúng doctrine, hết PUT tạm:

PATCH /branches/:id/founder — gán Founder, có set members.branch_id = chi này

PATCH /branches/:id/origin — trở Origin, không đổi branch_id người đó

PATCH /members/:id/branch (hoặc batch) — lắp hộ vào chi

Không tạo member ma; trùng tên chỉ cảnh báo

Không có lát này thì Submit chỉ sống nhờ SQL/PUT thủ công.

2. Approve ghi quota

Body { note, max_generation_span }. Tem + ghi N vào cột Int. Chỉ dẫn Admin admin_note / payload ticket. Chưa siết 422 lúc gắn người.

3. Siết cửa tạo chi

USER mặc định cấm POST /branches (doctrine §4.1). Chỉ CLAN_ADMIN / SYSTEM hoặc USER có grant Founder (grant = nợ).

4. Sau đó (không lần 1-3)

Siết quota khi gán member · FE Admin (Origin, tem, lý do) · 4 đời login · EDITOR ≠ Founder · MERGED · cấm USER chi 2 · form duyệt.

Đề xuất mở một: Lát 1 (founder / origin / branch_id). Quota + siết POST để lát kế, tránh gói to.

Bảo «mở Lát 1» thì mới chốt hợp đồng 3 PATCH rồi sinh file.

Lát 1

Worked for 9s

Lát 1 = gán nhãn tổ chức. Không tạo member. Không đổi huyết thống. Không tem.

3 lệnh

Method Path Body Việc

PATCH /api/branches/:id/founder { member_id } branches.founder_id = member. Có ghi members.branch_id = chi này

PATCH /api/branches/:id/origin { member_id } branches.origin_member_id = member.

Không đổi branch_id người đó

PATCH /api/members/:id/branch { branch_id } hoặc { branch_id: null } Lắp / tháo một người vào chi

Ai: CLAN_ADMIN · SYSTEM_ADMIN · USER chỉ khi branches.changed_by === user và chi DRAFT | PROVISIONAL | REJECTED.

findUnique / update không nhét tenant_id. ALS giữ.

Rule

Founder

Member cùng tenant, chưa xóa.

Nếu members.branch_id đã trở chi khác 422 BRANCH_FOUNDER_TAKEN (không cướp).

branch_id null hoặc đã = chi này gán founder + set branch_id.

Founder = Origin (cùng người) được.

Origin

Member cùng tenant, chưa xóa.

Đã có branch_id chi khác vẫn cho. Không đưng cột đó.

origin_member_id null = gỡ tổ (chỉ khi chi chưa APPROVED / MERGED).

Lắp branch_id

Chi cùng tenant.

Người đã đứng chi khác 422 MEMBER_BRANCH_TAKEN (multi-home = nợ).

Sau gắn: nếu chi có max_generation_span chưa siết đời (lát 2).

Tháo branch_id: null

Chỉ khi chi DRAFT | PROVISIONAL | REJECTED.

SUBMITTED | UNDER_REVIEW | APPROVED | MERGED 409.

Tháo founder đang gắn: đồng thời founder_id = null nếu founder_id === member. Origin không xóa khi tháo (tổ không thuộc chi).

Không làm

POST member từ cửa chi · sửa father_id / gender / is_alive · ghi quota · form FE ·

silentEmit bắt buộc (có thể BPL BRANCH_MEMBER_ATTACH / không emit nếu muốn lát mỏng).

Đề xuất lát này: writeBpl BRANCH_MEMBER_ATTACH + BRANCH_EDITOR_* không.

Founder/Origin: BPL BRANCH_DRAFT_CREATE không hợp — dùng attach + ghi extraMetadata.role = FOUNDER|ORIGIN. Hoặc chưa BPL, chỉ DB.

File

Append branch.routes / controller / service.

Gắn members.branch_id: hàm trong branch.service (không sửa member.controller lớn) — route member đặt ở branch.routes PATCH /members/:memberId/branch để lịch mount.

Gọn hơn: cả 3 lệnh trên /api/branches:

PATCH /:id/founder

PATCH /:id/origin

PATCH /:id/members body { member_id, op: "ATTACH"|"DETACH" }

Không đụng member.routes.

Xác nhận 2 điểm rồi sinh zip:

Ba lệnh gắn trên /api/branches (không mở member.routes) — ok?

Lát này có writeBpl attach hay chỉ ghi DB?

OK

có writeBpl attach

Worked for 1m 20s

Lát 1 trong zip. Đề 3 file branch.* (giữ REVIEW). Restart BE.

Lệnh Body Hành vi

PATCH /api/branches/:id/founder { "member_id" } Gán founder và members.branch_id

PATCH /api/branches/:id/origin { "member_id" } Chỉ trả tổ, không đổi chi người đó

PATCH /api/branches/:id/members { "member_id", "op": "ATTACH"|"DETACH" } Lắp / tháo
Founder đã đứng chi khác 422 BRANCH_FOUNDER_TAKEN
ATTACH đã đứng chi khác 422 MEMBER_BRANCH_TAKEN
DETACH khi chi APPROVED (chi Xích) 409 — thử trên chi nháp
BPL BRANCH_MEMBER_ATTACH (SET_FOUNDER / SET_ORIGIN / ATTACH / DETACH)
Thunder trên chi nháp mới: origin (người đời cũ) founder (người chưa gắn) ATTACH
con + PUT member father_id submit. Chi đã APPROVED chỉ nên PATCH origin (trở),
không DETACH.

Chưa thấy zip file để download.

Worked for 11s

Zip Lát 1 đây — tải bên dưới, đề 3 file branch.* rồi push + restart BE.

Chưa có sẵn dữ liệu để smoke các tình huống.

Worked for 10s

Chi Xích đã APPROVED — không dùng để tháo. Tạo bộ nháp riêng. Tenant bạn đang
dùng: 81b984e0-0e21-4d64-a523-6b259cae1cad.

0. Người có sẵn (không clone)

Ghi ra giấy 3 id (GET /api/members lọc tenant):

Vai test Điều kiện

A — Tổ Đời cũ, đã có branch_id (vd. founder chi Xích 354f353a-...)

B — Người khai branch_id null (tạo mới nếu hết chỗ)

C — Con của B branch_id null, sẽ gán father_id = B

Nếu thiếu B/C — tạo member chuẩn (POST /api/members), không gửi branch_id.

1. Chi nháp

POST /api/branches

{ "name": "Chi nhap L1" }

Lưu id {{L1}}. Kỳ vọng PROVISIONAL / DRAFT.

2. Origin — không cướp chi A

PATCH /api/branches/{{L1}}/origin

{ "member_id": "" }

GET /api/members/ branch_id vẫn chi cũ.

GET /api/branches/{{L1}} origin_member_id = A.

3. Founder = B

PATCH /api/branches/{{L1}}/founder

```
{ "member_id": "" }
```

GET B branch_id = {{L1}}. Chi founder_id = B.

Thử founder = A (đã đứng chi khác) 422 BRANCH_FOUNDER_TAKEN.

4. ATTACH C + huyết thống

PATCH /api/branches/{{L1}}/members

```
{ "member_id": "", "op": "ATTACH" }
```

PUT /api/members/

```
{ "father_id": "" }
```

(cha-mẹ không nằm cửa Lát 1.)

ATTACH A vào {{L1}} 422 MEMBER_BRANCH_TAKEN.

5. DETACH rồi gắn lại

PATCH .../members { "member_id": "", "op": "DETACH" } C branch_id null.

Gắn lại ATTACH C.

6. Submit nháp (không đựng chi Xích)

POST /api/branches/{{L1}}/submit {} 200 nếu B founder + cặp B-C cùng chi.

Không có B/C?

POST /api/members tối thiểu (đúng schema form bạn đang dùng: full_name, gender, không phone unique). Hai người, rồi làm từ bước 3.

Không seed SQL tay trừ khi GET member không đủ — giữ ALS, không UPDATE tenant_id.

POST http://localhost:10000/api/members

body:

```
{
  "tenant_id": "81b984e0-0e21-4d64-a523-6b259cae1cad",
  "full_name": "Nguyễn Đình Xích",
  "gender": "NAM",
  "note": "Thân sinh ra 5 người con. Mai táng tại nghĩa trang quê nhà",
  "changed_by": "4863e56c-4ac4-412b-a7eb-c814e631453a"
}
```

Lỗi:

prisma:error

Invalid tx.members.create() invocation in

/Users/quang.nguyenthanh/QuanLyDongHo/gia-pha-

backend/src/modules/members/member.service.js:71:39

```

68 createdByMemberId = actorUser?.member_id || null;
69 }
70
71 const member = await tx.members.create({
data: {
tenant_id: "81b984e0-0e21-4d64-a523-6b259cae1cad",
current_address_id: null,
changed_by: "4863e56c-4ac4-412b-a7eb-c814e631453a",
created_by: "4863e56c-4ac4-412b-a7eb-c814e631453a",
created_by_member_id: "354f353a-d6f4-46de-ac6c-b8c18258bafc",
+ full_name: String
}
})

```

Argument full_name is missing.

Worked for 16s

Không phải thiếu tên trên mạng. POST /api/members không đọc full_name ở root.

Controller gởi cả req.body vào createFullMember. Service chỉ lấy:

```

const { memberData, currentAddr, biographyData, changed_by } = payload;
// create: { ...safeMemberData } chính là memberData
Bạn gửi phải memberData = undefined Prisma không thấy full_name.

```

tenant_id / changed_by trên body bỏ (ALS + actor lấy từ JWT).

Body đúng

```

{
"memberData": {
"full_name": "Nguyễn Văn B L1",
"gender": "NAM",
"note": "Nguoi khai chi nhap L1"
}
}

```

Tạo con:

```

{
"memberData": {
"full_name": "Nguyễn Văn C L1",
"gender": "NAM",
"note": "Con B — chưa gan chi"
}
}

```

father_id gắn sau bằng PUT /api/members/:id nếu update whitelist cho phép — updateFullMember không có father_id / branch_id. Quan hệ cha: SQL hoặc cửa khác. Lát 1 chỉ PATCH /branches/:id/members { op: ATTACH }.

Đừng đặt tên trùng «Nguyễn Đình Xích» nếu đã có hàng A — khó phân biệt lúc chọn Origin.

Hai id B/C rồi làm PATCH origin / founder / ATTACH trên chi nháp.

Smoke OK:

user login vẫn tạo được Chi -> submit.

user_login có thể ATTACH/DETACH member

ADMIN approve đã ghi được: "max_generation_span": not null

```
{
  "status": "success",
  "data": {
    "branch": {
      "id": "31b1e7c6-e70f-44cf-9b8e-bb9954700121",
      "parent_id": null,
      "name": "Chi nhà ông Nguyễn Hữu Xuyên & Đỗ Thị Tâm",
      "description": "Test L1",
      "headquarters_address_id": null,
      "social_groups": null,
      "contact_phone": null,
      "contact_email": null,
      "changed_by": "16413025-fd5d-4673-ae29-b8abfab373b0",
      "created_at": "2026-09-15T02:18:27.449Z",
      "updated_at": "2026-09-15T02:36:38.112Z",
      "deleted_at": null,
      "founder_id": "6dd633ff-0d21-4add-9e1d-50749b85df66",
      "tenant_id": "094243c5-97f3-4521-a74f-30f7164729f6",
      "status": "APPROVED",
      "onboarding_case_id": null,
      "origin_address_id": null,
      "origin_member_id": "ab8f78ee-1a0c-4627-9b3f-a7f6183dcff1",
      "max_generation_span": 4
    },
    "ticket": {
      "id": "ed65714e-bbf8-4e16-b4ef-c80c17df1ca3",
      "tenant_id": "094243c5-97f3-4521-a74f-30f7164729f6",
      "ticket_type": "BRANCH_REVIEW",
    }
  }
}
```

```
"status": "APPROVED",
"requester_user_id": "c736c73c-68c4-4a86-a668-f583c2d5dcfb",
"target_table": "branches",
"target_id": "31b1e7c6-e70f-44cf-9b8e-bb9954700121",
"payload": {
  "note": "Tôi tạo 3 đời trong chi.",
  "branch_id": "31b1e7c6-e70f-44cf-9b8e-bb9954700121",
  "branch_name": "Chi nhà ông Nguyễn Hữu Xuyên & Đỗ Thị Tâm",
  "from_status": "PROVISIONAL",
  "approver_note": "Đồng ý",
  "max_generation_span": 4
},
"payload_schema_version": 1,
"admin_note": "Đồng ý",
"reviewed_by": "16413025-fd5d-4673-ae29-b8abfab373b0",
"reviewed_at": "2026-09-15T02:36:37.909Z",
"applied_at": null,
"correlation_id": "ae7123a0-f2d0-4d93-a25b-38bd07235e63",
"created_at": "2026-09-15T02:34:11.939Z",
"updated_at": "2026-09-15T02:36:37.909Z",
"changed_by": "16413025-fd5d-4673-ae29-b8abfab373b0",
"deleted_at": null
}
}
}
```

2 / 2

Worked for 14s

Smoke Lát 1 + quota Approve khớp hợp đồng trên chi 31b1e7c6-....

Kiểm Kết quả

Origin ab8f78ee-... — tổ người, không phải địa

Founder 6dd633ff-...

Tem APPROVED + ticket ed65714e-...

Quota max_generation_span: 4 trên chi và payload / note Admin

USER ATTACH/DETACH chạy (chi nháp / trước tem)

from_status: PROVISIONAL trên ticket = snapshot lúc gửi — đúng.

Lịch doctrine — chưa khóa

USER login vẫn POST /branches + submit. Học thuyết §4.1: mặc định cấm tạo chi, chỉ ADMIN hoặc USER có grant. Đây là nợ đã biết, không phải fail smoke Lát 1.

Quota đã ghi, chưa siết: ATTACH đời thứ 5 trên chi N=4 vẫn qua (lát 2).

Việc tiếp (một lát)

Siết quota lúc ATTACH — 422 nếu vượt max_generation_span (cần rule tính đời rõ: so Origin / Founder / members.generation).

Khóa USER POST /branches — đúng §4.1.

FE chọn Origin / Founder / tem + hiện N.

Đề xuất: (1) siết quota nếu cây đã có generation; (2) nếu muốn sạch cửa tạo trước. Bảo một số rồi mới sinh mã.

Về vấn đề: Siết quota lúc ATTACH — 422 nếu vượt max_generation_span (cần rule tính đời rõ: so Origin / Founder / members.generation).

Tôi nhận thấy một số vấn đề sau:

a) Chỉ member với user login, gọi tắt là MWL (Member With Login) mới có quyền tạo member khác trong MFO.

b) Founder (người quản lý tổ chức)/Trưởng Chi (bất kỳ ai được gán quyền thay đổi) bắt buộc là MWL.

Nota: a) và b) đảm bảo giám sát được sự thay đổi dữ liệu trên audit_logs.

c) Phải có khuôn mẫu bắt buộc MWL tuân thủ khi tạo MFO:

Dòng 1 :Ông/bà nội nội tộc

Dòng 2: Cha/Mẹ nội tộc (Con dâu phải khai cha/mẹ chồng, con gái khai cha/mẹ đẻ,...)

Dòng 3: Con (Con đẻ, con nuôi, con dâu, con rể, con đỡ đầu, con ngoài giá thú,...)

Dòng 4: Cháu

Dòng 5: ChắtCách tính max_generation_span tính từ

Worked for 25s

Đoạn cuối cắt: «Cách tính max_generation_span tính từ» — chưa có mốc. Dưới đây chốt a-c rồi đề mốc tính.

a) Chỉ MWL tạo người trong MFO

Đúng cửa «Gia đình tôi». Audit cần changed_by = **users.id** và created_by_member_id = users.member_id.

Không khóa ADMIN tạo người lịch sử trên chi tổ chức (đời ≤ n). Đó không phải MFO.

POST /api/members hiện nhận JWT đã gán MWL. Thiếu: chặn user không có users.member_id.

b) Founder / người được quyền sửa chi = MWL

Khớp tách Tổ ≠ Người khai:

Vai Login?

Origin Không bắt — người chết / đời cũ

Founder / Trưởng chi / EDITOR chi Bắt MWL

Hàng 354f353a / ab8f78ee làm Origin được. Làm Founder thì 422 nếu users.member_id không trở tới họ.

Không đụng users.status. Grant EDITOR ≠ khóa tài khoản.

c) Khuôn 5 dòng — cửa MFO, không phải cửa Chi

Đó là mẫu khai hộ, không phải độ sâu cây chi.

Thiếu dòng «tôi» (MWL). Năm dòng họ liệt kê là họ hàng quanh tôi, không phải 5 tầng generation.

Gợi ý khuôn MFO (trực hệ):

Dòng Quan hệ Bắt?

0 Tôi (MWL) có sẵn login

1 Ông/bà nội tộc (Origin hộ) 1 người neo

2 Cha/mẹ nội tộc ≥ 1

3 Con (+ dậu/rể/nuôi... ghi child_type + Note) tùy

4 Cháu tùy

5 Chắt tùy, và dừng nếu N chi không đủ

Con dậu khai cha/mẹ chồng; con gái khai cha/mẹ đẻ — VoiceHelp, không nhét vào father_id sai họ.

Khuôn này không thay max_generation_span của chi. Chi N=4 có thể chứa nhiều hộ; mỗi hộ MFO bị cắt theo N của chi đang gắn.

Mốc max_generation_span — phải chọn một

Ba cách, không trộn:

P1 — Từ Origin chi

Đời Origin = 0. Gắn người có khoảng cách huyết thống (theo father_id/mother_id) > N 422.

Khớp tem Admin «chi được phép sâu N đời từ tổ».

P2 — Từ Founder / MWL

Đếm lên ông và xuống cháu. N=4 ≈ ông...cháu hoặc cha...chắt — phải ghi rõ.

P3 — Từ cha/mẹ tôi (bản doctrine cũ «4 đời từ cha/mẹ»)

Origin hộ = cha; ông là +1 lên.

Smoke Admin ghi N=4 trong khi mẫu c) có 5 dòng + tôi nếu không chốt mốc, 422 sẽ lệch kỳ vọng.

members.generation hiện không đủ tin (nhiều hàng null, suy luận cây chỉ local). Siết ATTACH nên đếm bước cha/mẹ trên đồ thị, không tin cột generation một mình.

Trả lời giúp 2 câu rồi mới siết code:

max_generation_span tính từ Origin chi / Founder / cha-mẹ MWL?

N=4 nghĩa là 4 bước (Origin + 3 đời dưới) hay 4 dòng khuôn (ông...cháu, không tính tôi)?
Khuôn mẫu (5 dòng đời) nói trên dành cho bất kỳ thành viên nào là MWL, thuộc 1 trong 5 dòng nói trên, thực hiện khai báo. Ví dụ, nếu MWL thuộc Dòng 0 thì có thể khai MFO tính từ chính mình cho maximum 4 dòng dưới. Nếu MWL thuộc Dòng 3 thì khai cho các Dòng còn lại.

Quy tắc: Dòng nào chưa biết thì để trống. Đời được suy luận từ Origin. Origin chính là Dòng 0.

Làm thế nào để xác định được đời của Origin? Khi trình ADMIN -> ADMIN buộc phải xác định được đời của Origin và buộc phải xác định Chi mà MFO thuộc về. Nếu không thì không phê chuẩn. Khi đã Approved thì mọi thông tin đã chắc chắn. Nếu có sai lệch thì ADMIN phải tự sửa. Như vậy, MWL khai báo bắt buộc phải theo cấu trúc cây 5 Dòng (thể hiện trên form). max_generation_span luôn =5.

Như vậy có ổn không? Còn vướng vấn đề gì không?

Worked for 25s

Ổn nếu 5 dòng là cửa sổ form MFO, không phải độ sâu mọi chi. Còn vài chỗ tự đụng nhau.

Chỗ đúng

Dòng neo Origin = Dòng 0. Đời người khác = khoảng cách huyết thống từ Origin, không tin generation rỗng.

Ô trống được. Không bịa đời.

ADMIN lúc duyệt bắt (1) đời Origin trên sổ họ, (2) chi chứa MFO. Không đủ thì không tem.

Founder/sửa chi = MWL audit changed_by.

Sai sau tem: ADMIN sửa, không mở USER phá khung.

Mâu thuẫn cần chốt

1. Origin = Dòng 0 nhưng «MWL thuộc Dòng 0»

Origin thường là ông/cụ không login. MWL hầu như không đứng Dòng 0.

Đọc lại cho khớp: 5 dòng gắn Origin, MWL rơi vào một dòng 0...4. Đúng Dòng 0 chỉ khi MWL chính là tổ (hiếm).

2. max_generation_span luôn = 5

Cột vừa smoke N=4 trên chi Xuyên. Nếu luôn 5 thì cột thừa, Admin ghi 4 thành sai.

Tách hai số:

Số Nghĩa

Cửa sổ MFO Form luôn 5 dòng (D0...D4), cố định UX

max_generation_span trên chi Trần Admin cho chi đó (4 / 5 / 8...). Form không được khai dòng > N

N chi < 5 form ẩn dòng dưới. N > 5 MFO chỉ khai 5 dòng/lần; nói cửa sổ = lát khác.

3. «Approved thì chắc chắn» vs PUT tự do

Đã đóng: tem ≠ niêm field. Giữ: USER MFO không phá khung sau tem; ADMIN vẫn PUT. Đừng nói «mọi field đóng».

4. Một chi, nhiều MWL

Hai người khai cùng cửa sổ 5 dòng trùng ông/cha. Chưa có rule merge (chọn member có sẵn / 422 trùng tên). Nợ, không chặn lát form.

5. Dòng hiển thị ≠ father_id

Con dâu «Dòng 3» không phải con huyết thống Origin. Cây siết quota phải đi huyết thống (cha/mẹ đẻ). Dâu/rể/nuôi = child_type + Note, không tính bước đời.

6. Đời Origin lấy từ đâu

ADMIN nhập đời tuyệt đối trên sổ họ (vd. đời 7) vào members.generation của Origin lúc duyệt — một lần.

Dòng form = generation(person) - generation(Origin).

Thiếu số này 422, đúng ý «không phê nếu chưa biết đời Origin».

Luật gọn (đề xuất)

Form MFO = 5 ô D0...D4, D0 = Origin đã chọn.

MWL chỉ sửa ô trống + người mình tạo; không đề người đã APPROVED trừ ADMIN.

Trần chi = max_generation_span (Admin ghi lúc tem). Gắn người có generation(Origin) + span - 1 bị vượt 422.

Submit MFO thiếu Origin hoặc chưa chọn chi 422; ADMIN không đoán.

USER không POST /branches (nợ §4.1 vẫn mở).

Vướng còn lại (không chặn form)

Map dâu/rể vào dòng mà không làm hỏng đếm đời

Trùng người khi hai MWL khai

generation Origin chưa có cột riêng — dùng members.generation

Chi lịch sử sâu hơn 5 đời vs form 5 ô

Grant Founder = MWL chưa kiểm tra users.member_id

Trả lời một câu: N trên chi có được khác 5 không?

Có giữ cột, form 5 ô cắt theo N.

Không xóa ý nghĩa quota, luôn 5, Admin không còn «cho phép 4 đời».

Tôi nhận thấy từ thực tiễn như sau:

Một founder hoặc là người nắm rõ một Chi, một cây họ nào đó hoặc phải là người nắm rõ về gia đình nhà mình (thường là 3 đời - 5 đời). Do vậy, cho dù founder có biết rất nhiều nhánh từ một cây gốc nào đó nhưng mỗi lần trình chỉ được phép tạo một cây với độ sâu 5 dòng. Nếu muốn tiếp tục tạo tiếp thì trình tiếp đến lúc nào muốn dừng thì thôi. Mỗi lần trình mới đều được ADMIN giám sát chất lượng và sự đúng đắn của một cây đã tạo ra. Tóm lại, 2 loại trình: Trình dự kiến và trình phê chuẩn kết quả. Bất kỳ trình dự kiến nào cũng chỉ cho phép max_generation_span = 5. Điều mô tả ở trên có nghĩa là ADMIN có thể cho phép một Founder (phải là MWL) nào đó (có thể chính là ADMIN/TRƯỞNG HỌ/TRƯỞNG) tạo lập các MFO (cả theo chiều sâu lẫn theo chiều ngang) nhưng mỗi lần tạo lập phải được phép của ADMIN và kết quả phải được ADMIN dán tem. Chưa nghiệm thu việc cũ thì không thể làm được việc mới.

Liệu có ổn không? Còn những issues gì khác không, bạn ơi?

Worked for 40s

Hướng ổn nếu đơn vị làm việc là một lần trình 5 dòng, không phải «cả chi mãi mãi sâu 5». Còn vài cửa dễ kẹt.

Chỗ đúng

Founder giới một cây hoặc họ mình — không bắt biết cả họ.

Mỗi lần chỉ 5 dòng Admin đọc nổi, hết rác một phát 20 đời.

Muốn sâu/ngang thêm ticket mới, tem lần trước xong đã.

Hai cửa: xin làm rồi nộp kết quả.

Founder = MWL.

Chưa nghiệm thu thì không mở việc kế — một Founder một việc mở.

max_generation_span = 5 lúc này = tràn một ticket, không còn là tràn cả chi. Chi đã tem N=4 hôm nay khác nghĩa — cần đổi tên trong đầu (và lát sau: cột / payload ticket).

Issues còn lại

1. Phạm vi «việc cũ chưa xong»

Khóa theo Founder? theo chi? theo tenant?

Theo Founder: không làm song song họ anh và họ mình.

Theo chi: chi khác vẫn mở được.

Nên: một ticket mở / một Founder / một chi. Chi khác hoặc Founder khác không bị chặn.

2. Ai duyệt việc của ADMIN

Trường hợp tự trình + tự tem = mất giám sát.

Rule: người reviewed_by $\neq$ requester_user_id, trừ khi tenant chỉ có một SYSTEM_ADMIN (ghi Note).

3. Ngang vs sâu

5 dòng = sâu. Ngang (5 con, 2 vợ) không nằm trong số 5.

Một ticket: sâu ≤ 5 và số người mới $\leq$ trần mềm (vd. 20) — nếu không, một lần «5 dòng» nhét 80 cháu.

4. Hai loại trình $\neq$ hai status hiện có

Máy hiện tại: DRAFT SUBMITTED APPROVED.

Cần đủ:

Việc Ý

Dự kiến Xin cửa sổ: Origin, chi, «tôi đứng dòng nào», sâu ≤ 5

Làm DRAFT/PROVISIONAL — gán người trong cửa sổ đã cho

Kết quả Submit lần 2 — Admin tem + generation Origin

Một BRANCH_REVIEW với payload.kind = PLAN | RESULT hơn là thêm enum lớn. PLAN chưa được tem thì không ATTACH ngoài khung.

5. Nối ticket 2 vào ticket 1

Ticket 2 sâu tiếp (D5 thành Origin cục bộ) hay ngang (MFO khác cùng chi)?

Phải ghi trên PLAN: mode: DEPTH | WIDTH + anchor_member_id (người D4 lần trước hoặc anh/em). Không ghi thì cây gãy.

6. Tem chi vs tem việc

Chi APPROVED hôm nay = «chi được công nhận».

Việc 5 dòng APPROVED = «lô người này được nhận».

Đừng gộp: chi có thể APPROVED sớm (có Origin + Founder + 1 cặp) rồi các lô MFO sau vẫn PLAN RESULT. Hoặc chi chỉ tem khi lô đầu nghiệm thu. Chọn một, kéo 409 «đã APPROVED không submit».

7. Sửa lô đã tem

«Admin tự sửa» — USER không đụng người đã RESULT-APPROVED. DETACH sau tem chi vẫn 409 (đúng lát 1).

8. Grant Founder vs từng PLAN

«ADMIN cho phép Founder tạo MFO» + «mỗi lần phải xin» = grant tư cách một lần, mỗi lô một PLAN. Grant không thay ticket.

Tóm lại

Ổn nếu chấp nhận: 5 = kích thước một lô, chi có thể nhiều lô; việc mở serialize theo Founder+chi; PLAN ≠ RESULT; ngang không ăn số 5; người duyệt ≠ người trình.

Chưa ổn nếu giữ branches.max_generation_span = trần cả đời chi và = 5 mọi ticket — hai nghĩa một cột.

Trả lời giúp 2 điểm rồi mới đụng máy trạng thái:

Chi đã APPROVED còn nhận lô MFO mới (PLAN+RESULT) không?

«Khóa việc cũ» theo Founder+chi hay theo cả tenant?

Để giải quyết các vấn đề, tôi đề nghị thế này:

Trình để được phép (trình lần 1) tạo MFO bắt đầu từ Origin -> Người trình nếu rõ Origine là ai, thông tin cụ thể. Không chấp nhận Origin= Unknown Member (UM) khi trình. Có thể, chỉ biết Tôi, con, cháu thì Dòng 0 chính là Tôi (cha/mẹ mất sớm nên không rõ thông tin). Nếu Trình lần 1 được phê chuẩn -> Founder tạo đúng Tôi, con, cháu (không nhất thiết đủ 5 Dòng); miễn là Origin phải rõ ràng.

Thường ADMIN khác Trưởng họ, Trưởng chi. Vì vậy, việc trình là trình Trưởng họ / Trưởng chi. ADMIN là người thao tác và xử lý hệ thống IT. Tuy nhiên, Trưởng họ / Trưởng chi lại là người chỉ nắm rõ thông tin về Họ/Chi. Do vậy, cần nội dung trình mà cả hai đều đọc được thì Trưởng họ / Trưởng chi nhắc cho ADMIN thao tác giúp.

Vấn đề dọc/ngang sẽ không bị vướng. Nếu mỗi lần tạo dựng một MFO (theo chiều sâu maximum 5). Nếu muốn tạo dựng MFO theo chiều ngang thì luôn phải chỉ rõ Origin.

Có hợp lý? Còn vướng mắc gì không nhỉ?

Worked for 18s

Hợp lý hơn bản trước. Đơn vị làm = một MFO neo một Origin đã biết, sâu ≤ 5, thiếu dòng được.

Chỗ đóng được

Ý Kết

Cấm Origin = UM lúc trình lần 1 Đúng — không có neo thì không tem

Chỉ biết tôi-con-cháu Dòng 0 = tôi Đúng — Origin họ = MWL

Sau khi lần 1 được phép: tạo đúng những người đã xin, không bắt đủ 5 Trần 5, sà = Origin + ≥1 người hoặc ít nhất cây đã kê

MFO ngang = MFO mới, phải chỉ Origin Hết mớ hồ dọc/ngang
Trưởng họ/chi nằm đúng sai; ADMIN IT bấm máy Đúng đời thực
Vướng còn lại (gọn)

1. Trong phần mềm, «Trưởng họ» đang là ai?

Hiện chỉ có USER / CLAN_ADMIN / SYSTEM_ADMIN. Chưa có role Trưởng họ / Trưởng chi.

V1 thực tế: một người mang CLAN_ADMIN = Trưởng (duyet đúng sai). ADMIN IT = cùng role hoặc SYSTEM bấm hộ.

«Nhắn Zalo rồi ADMIN bấm» ngoài hệ thống — không dựng workflow 2 người trong lát này. Audit vẫn là reviewed_by = user bấm Approve.

2. Origin lần 1 phải là member đã có

«Thông tin cụ thể» ≠ tên gõ tự do. Phải chọn member_id (tạo sẵn hàng Origin trước, hoặc chính MWL). Không chọn được 422, không sinh UM.

3. Lần 2+ (MFO ngang / sâu tiếp)

Origin = member đã có trên sổ (thường đã nằm lô đã tem). Không Origin mới kiểu UM.

Cùng Origin + thêm con = ngang. Origin = người Dòng 4 lô trước = sâu tiếp.

4. Chi đã APPROVED

Mỗi MFO mới = ticket mới (PLAN rồi RESULT), không đi lại /approve của chi (409).

Chưa nói trong máy hiện tại — lát ticket kind / type mới, không phá tem chi.

5. Đời tuyệt đối của Origin

Dù Dòng 0 = «tôi», Trưởng/Admin vẫn gán members.generation lúc duyệt lần 1. Không gán không suy được Dòng 1...4.

6. Tự duyệt

User vừa là Founder vừa là CLAN_ADMIN (họ nhỏ) tự tem được. Chấp nhận + admin_note «tự duyệt». Họ lớn: người trình ≠ người bấm.

7. Trùng người giữa hai MFO

Hai lô cùng Origin khai trùng một con chọn hàng có sẵn, không tạo bản thứ hai (cảnh báo tên — nợ).

Không còn vướng khái niệm dọc/ngang nếu mọi lô đều có Origin cụ thể và sâu ≤ 5. Vướng còn lại là role duyệt (gộp CLAN_ADMIN = Trưởng cho V1) và tách tem chi ≠ tem lô MFO.

Lát kỹ thuật kế: không siết quota ATTACH mù; thêm payload.kind = PLAN|RESULT + bắt origin_member_id lúc PLAN. Bảo mở lát đó thì mới sinh mã.

Trong phần mềm, «Trưởng họ» đang là ai?

Trong schema members đã có:

```
enum members_roles {  
  TRUONG_TOC  
  TRUONG_NGANH  
  TRUONG_CHI  
  THANH_VIEN  
  TRUONG_HO  
  KHAC  
}
```

Nhưng thực ra phải được quản lý ở schema riêng. Nhưng không sao vẫn có thể mở rộng như bảng member_residences để quản lý lịch sử thay đổi members.role.

Origin lần 1 phải là member đã có: Đúng vậy. Nếu chưa có thì ADMIN phải tạo trước.

Lần 2+ (MFO ngang / sâu tiếp): Nếu chưa có thì ADMIN tạo, Có thể UM (nhưng phải do ADMIN tạo) để tránh rỗng cây. Đây là điều hay xảy ra trong thực tiễn. Do đó, chắc chắn phải định nghĩa UM với mã quy ước (thường kết hợp với generation để làm tính duy nhất).

OK như đã mô tả.

Đời tuyệt đối của Origin phải biết lúc trình lần 1.

Tự duyệt: Đúng, tự chịu trách nhiệm.

Trùng người giữa hai MFO. Mỗi MFO là theo huyết thống thì làm sao có chuyện trùng được?

Worked for 18s

Bổ sung khớp thực tiễn. Còn đúng một lớp nhằm: chức danh trên members ≠ quyền bấm máy.

1. members.roles ≠ người duyệt JWT

Enum đó là chức trên sổ (Trưởng tộc / họ / ngành / chi).

Cửa API vẫn là users.role (USER / CLAN_ADMIN / ...).

Duyệt PLAN/RESULT = MWL có members.role {TRUONG_HO, TRUONG_TOC, TRUONG_NGANH, TRUONG_CHI} và users.member_id trở đúng hàng đó — hoặc CLAN_ADMIN/SYSTEM bấm hộ (đúng ý «nhấn ADMIN IT»).

members.role một cột = chức hiện tại. Lịch sử chức bảng như member_residences (nợ, không làm lúc này). Không nhét lịch sử vào audit_logs schema.

EDITOR chi / Founder / members.role / users.status vẫn bốn việc khác nhau.

2. Origin lần 1 = member có sẵn

ADMIN tạo trước nếu thiếu. Không UM lúc USER trình lần 1.

3. UM — chỉ ADMIN, lô 2+

Được, để khỏi gây cây. Cần quy ước kéo nổ unique:

Một hàng member thật: full_name kiểu UM-Đ{generation}-{branch_short} hoặc

unknown_kind + generation + branch_id

members.generation bắt buộc (ADMIN gán)

Không login, không Founder, không Origin lần 1 USER

Khi biết tên: sửa cùng hàng (đổi tên), không tạo người thứ hai rồi bỏ UM

«Unique generation» toàn tenant thì sai (nhiều UM cùng đời khác chi). Unique gợi ý:

(tenant_id, branch_id, generation, um_slot) — cột um_slot hoặc Note. Chưa thêm unique DB nếu chưa migrate.

4–6. Tem lô / đời Origin lúc PLAN / tự duyệt

Giữ như đã chốt.

7. Trùng MFO huyết thống — vẫn xảy ra

Mỗi MFO là cửa sổ 5 dòng neo một Origin, không phải phân hoạch rời.

Tình huống Trùng

Hai anh em MWL, cùng Origin = cha Cha + ông xuất hiện hai lô

Con tạo MFO Origin = mình; cha đã khai con trong lô trước Cùng một member_id

Cháu và chú cùng khai một ông Origin / Dòng 1 trùng

Huyết thống không làm hai cửa sổ disjoint.

Rule: không INSERT người mới nếu đã có member_id; form chọn hàng có sẵn. Trùng tên chưa chắc trùng người (hai UM đời 5) — chỉ ADMIN gộp.

Dâu/rể không thuộc MFO huyết thống không vào đếm đời; không phải case trùng cửa sổ.

Còn mở (không chặn viết doctrine): map members.role được duyệt ticket; quy ước mã UM; ticket PLAN|RESULT tách tem chi.

Chưa sinh bảng member_role_history / unique UM cho đến khi bảo mở migrate.

Tôi muốn tìm hiểu kỹ:

7. Trùng MFO huyết thống — vẫn xảy ra.

Nếu xin phép ADMIN cho phép tạo từ ông tôi làm Origin với generation đã xác định thì không được tạo mà chỉ được chọn. Khi chọn ông tôi tức là có con của ông gắn kèm. Do đó có thể chọn cả hai trong mô tả MFO của tôi. Việc chọn trùng ông và bố là tốt chứ sao?

Tránh dư thừa là hợp lý chứ nhỉ?

Worked for 10s

Đúng. «Trùng» ở đây không phải lỗi — là cùng một người đứng nhiều cửa sổ.

Tạo vs chọn

PLAN: Origin = ông, generation đã có chỉ chọn member_id ông.

ADMIN đã tạo ông (và thường đã có bố) thì USER không POST thêm hàng «ông tôi».

Chọn ông không sinh bản sao. Form MFO của tôi tham chiếu cùng id.

«Có con ông gắn kèm» — nên hiểu là danh sách chọn

Khi chọn ông: hệ thống gợi ý các con đã có (father_id/mother_id = ông) — bố tôi, bác, chú.

USER tick người nào vào cửa sổ 5 dòng của mình.

Không tự:

gán hết con cháu ông vào branch_id MFO mới

tạo hàng con còn thiếu

Thiếu bác để trống hoặc chờ ADMIN / lô khác. Đó là «dòng chưa biết thì trống».

Chọn trùng ông + bố là tốt

Lô Origin Người (cùng id)

Cha tôi đã khai trước Ông ông, bố, ...

Tôi khai Ông ông, bố, tôi, con tôi

Hai lô chung ông và bố = một sổ, hai góc nhìn. Audit rõ ai khai thêm đời dưới. Không dư hàng.

Xấu chỉ khi: cùng tên «Nguyễn Văn A» mà insert hàng mới. Cửa chọn + cảnh báo trùng tên là đủ.

Tóm

Origin / bố đã có chọn, không tạo.

Con của Origin gợi ý, không auto-ATTACH cả nhánh.

Cùng member_id trên nhiều MFO = đúng.

Dư thừa = hai UUID một người — cái cần chặn.

Lát form: picker Origin + picker «người đã là con Origin», không createFullMember cho

Dòng 0-1 nếu đã có id.

Việc quyết định duyệt cho phép ai thừa hưởng dữ liệu đã có là do người phê duyệt chỉ định, như là một hướng dẫn ban đầu. Làm xong mà không đúng chỉ dẫn thì không phê duyệt. Vậy có ổn không nhỉ?

Worked for 4s

Ổn — nếu chỉ dẫn nằm trên ticket PLAN, không nằm ở tin nhắn ngoài sổ.

Cách hiểu

Phê lần 1 không chỉ «được làm», mà ghi:

Origin (member_id + đời)

Chi

Những id được phép dùng lại (ông, bố, ...)

Những dòng được phép tạo mới

Founder / MWL được giao

Làm xong (RESULT): đúng chỉ dẫn tem. Lệnh (tạo thêm ông mới, kéo người ngoài danh sách, đổi Origin) không tem, Note rõ.

Đó là giám sát chất lượng từng lô, khớp «chưa nghiệm thu thì chưa việc mới».

Điều kiện cho khỏi vỡ đoán

Payload PLAN có reuse_member_ids[] + allow_create_lines[] (vd. D2-D4).

Người duyệt đọc cùng màn hình USER đã gửi — Trưởng họ duyệt đúng sai; ADMIN IT chỉ bấm nếu được nhờ.

Máy có thể 422 sớm: tạo member không nằm allow_create, hoặc ATTACH id ngoài reuse + allow-create. Không bắt buộc hết luật ngay — V1 Trưởng đọc tay cũng được.

Chỉ dẫn là giao việc, không chuyển sở hữu người. Ông/bố vẫn là một hàng; «thừa hưởng» = được tham chiếu, không copy số.

Không ổn nếu

Chỉ dẫn chỉ nói miệng / Zalo — RESULT không có chuẩn để so.

«Thừa hưởng» hiểu thành clone member.

Không tem vì «không thích» mà không ghi lệnh chỉ dẫn nào.

Chốt: duyệt = giao phạm vi dữ liệu có sẵn; nghiệm thu = đối chiếu phạm vi đó. V1 ghi trên PLAN là đủ; siết 422 tự động là lát sau.

Vấn đề bây giờ là chỉ dẫn. Vậy dựa vào khung chuẩn 5 Lines. Ví dụ, line 0: X (gán luôn),

Dòng 1: Tạo mới Cha + mẹ đẻ. Dòng 2: ... Theo cách, gán sẵn, chỉ rõ chỗ nào tạo mới,...

Generation + Branch được gán khi phê duyệt.

NHư vậy có ổn không?

Worked for 4s

Ổn. Chỉ dẫn = khung 5 dòng trên PLAN, mỗi ô một việc.

Mẫu ô

Dòng Việc Ví dụ

0 Gán sẵn Origin member_id = X (bắt chọn, không UM)

1 Tạo mới / gán / trống «Tạo cha + mẹ đẻ»

2...4 Tạo / chọn / trống «Chọn tôi (MWL); tạo 2 con»

Mỗi ô chỉ một trong: ASSIGN (id có sẵn) · CREATE (loại: cha, mẹ đẻ, con...) · EMPTY.

Không ô nào «tùy hứng». RESULT đối chiếu đúng 5 ô đó.

Generation + Branch

Kê lúc trình lần 1: USER/Trưởng đề xuất chi (nếu biết) + đời Origin (bắt buộc biết — đã chốt).

Chốt lúc phê PLAN: Admin/Trưởng ghi origin.generation + branch_id vào payload đã duyệt.

Sau tem PLAN, Founder mới ATTACH/CREATE trong khung. Sai chi / sai đời không mở làm.

Không đợi phê mới «đoán» đời Origin. Phê là xác nhận số đã kê, có quyền sửa số rồi mới cho làm.

V1 đủ dùng

Payload PLAN:

```
lines[0...4]: { op: ASSIGN|CREATE|EMPTY, member_id?, hint }
```

origin_member_id

proposed_generation

proposed_branch_id

granted_generation // ghi lúc duyệt

granted_branch_id

Form 5 dòng + hai cột «gán / tạo» là đúng chỗ VoiceHelp. Siết 422 từng ô = lát sau; lúc này Trưởng đối chiếu tay khi RESULT.

Như vậy cần tóm lược tất cả những gì đã thống nhất để có thể lập kế hoạch công việc và tiến hành codeing.

Worked for 43s

Bản đóng để lập kế hoạch — file đủ để copy vào CM-Docs:

Đã live (đừng làm lại)

Chi CRUD + cycle · enum/BPL/notify · Origin + quota cột · SUBMIT/APPROVE/REJECT (tem chi, PUT field vẫn mở, 409 khi duyệt lại) · Lát 1 founder/origin/ATTACH · smoke USER tạo chi + ATTACH + Admin ghi max_generation_span.

Đã thống nhất, chưa code

MFO = cửa sổ huyết thống 5 dòng, Origin = Dòng 0, ô trống được.

Lô ≠ chi. Mỗi lô sâu ≤ 5; ngang/sâu tiếp = lô mới + Origin cụ thể.

PLAN rồi RESULT. Chưa nghiệm thu (theo Founder+chi) thì không lô mới.

Chỉ dẫn = 5 ô ASSIGN | CREATE | EMPTY. Đời Origin + chi kê lúc PLAN, chốt lúc phê.

Origin lần 1 = member có sẵn; USER không UM. UM chỉ ADMIN.

Chọn, không clone. Gợi ý con Origin, không auto-ATTACH cả nhánh.

Founder = MWL. Trưởng* = members.role; bấm máy = JWT Admin. Tự duyệt = tự chịu.

Tem chi ≠ tem lô.

Nợ / không lẫn PR

USER vẫn POST chi · cột max_generation_span chi vs trần lô 5 · lịch sử members.role · unique UM · 422 tự động từng ô · multi-home · 4 đời login.

Lát code đề xuất

Docs ticket PLAN|RESULT cửa PLAN khóa ATTACH trong khung RESULT picker FE siết POST chi / Founder=MWL.

Mở từng lát, hỏi rồi mới sinh mã. Q1 giữ máy Chi hiện tại.

Có lẽ phải nhấn mạnh form 5 lines (gọi tắt 5L) với tên gọi “Sơ đồ các đời” (cần các trạng thái tương ứng cho hai lần trình có thể lặp lại). Theo đó, Người trình căn cứ vào 5L, Thực hiện cũng theo 5L đã duyệt, Duyệt cũng căn cứ vào 5L để nghiệm thu và duyệt.

Liệu như vậy có lý không?

Worked for 4s

Có lý. Một khung 5L = «Sơ đồ các đời» làm chuẩn chung cho trình / làm / duyệt, và lặp được, thì khỏi lệch chỉ dẫn.

Vì sao ổn

Cùng một 5L: người trình kê, người làm đúng 5L đã duyệt, người duyệt đối chiếu 5L.

Lặp: hết lô này 5L mới (sâu tiếp hoặc ngang, Origin rõ).

Trống ô = chưa biết, không bịa.

Origin lần 1 không UM; UM chỉ ADMIN, lô sau, để khỏi gãy cây.

Máy trạng thái 5L (lặp)

Mỗi lô 5L là một vòng, có thể nhiều vòng trên cùng chi:

5L_DRAFT soạn sơ đồ (chưa gửi)

5L_SUBMITTED trình lần 1 (xin phép theo 5L)

5L_PLAN_OK được phép làm đúng 5L đã duyệt

5L_RESULT nộp kết quả (cây đã dựng theo 5L)

5L_APPROVED nghiệm thu — đóng lô

5L_REJECTED không đạt sửa 5L hoặc 5L mới

5L_WITHDRAWN hủy lô / thay bằng ticket mới

Sau 5L_APPROVED hoặc 5L_REJECTED (nếu bỏ lô): được mở 5L mới (ngang hoặc sâu tiếp), Origin phải rõ.

Không mở lại SM cũ. Không thêm branch_status. Trạng thái 5L nằm ở ticket (payload / status hiện có của proposals).

Chỉ dẫn trên 5L

Mỗi dòng ghi sẵn:

Gán: member_id (ông/bố đã có)

Tạo mới: loại quan hệ + mô tả

Trống: chưa biết

generation Origin + branch_id chốt lúc phê PLAN. Làm ngoài chỉ dẫn không duyệt RESULT.

Vướng còn

Map Trưởng* (members.role) ai được bấm duyệt vs ADMIN IT — V1 có thể gộp CLAN_ADMIN.

UM do ADMIN: mã + đời + chi; chưa unique DB.

Nối lô: DEPTH | WIDTH + anchor_member_id phải có trên PLAN.

Cột max_generation_span chi vs trần lô 5 — hai nghĩa; lát sau tách.

Hai MWL cùng ông: chọn trùng là tốt; cấm insert trùng người.

Form FE 5L + VoiceHelp chưa có.

Việc tiếp khi bảo: đưa file này vào docs, rồi lát ticket PLAN | RESULT (chưa form FE). Không sinh mã trước khi hỏi.

TRƯỞNG HO/TRƯỞNG CHI/CLAN_ADMIN có quyền duyệt. TRƯỞNG HỌ có thể uỷ quyền cho TRƯỞNG CHI phê duyệt cho từng chi nếu MFO thuộc chi mà họ quản lý. Nếu chưa chỉ định TRƯỞNG HỌ/TRƯỞNG CHI thì CLAN_ADMIN quyết định tất. Chúng ta đã có các schema để làm uỷ quyền rồi, đúng vậy không? Bạn review schema.prisma trên repos mà tôi vừa update.

Worked for 13s

Ổn với điều kiện: chỉ dẫn = 5 dòng trên PLAN; đời + chi kê lúc trình, chốt lúc phê. File tóm tắt để lập kế hoạch:

Đã live — đừng làm lại

Chi CRUD + cycle · enum/BPL/notify · Origin + quota cột · SUBMIT/APPROVE/REJECT (tem chi, PUT field mở, 409 khi duyệt lại) · Lát 1 founder/origin/ATTACH · smoke: USER tạo chi + ATTACH, Admin ghi max_generation_span + Origin.

Đã thống nhất — chưa code hết

Đơn vị: Chi = tổ chức. MFO = cửa sổ 5 dòng huyết thống, Origin = Dòng 0. Lô = một lần xin-làm-tem. Nhiều lô / chi.

Hai trình: PLAN (xin phép) rồi RESULT (nghịệm thu). Chưa nghịệm thu (theo Founder + chi) thì không lô mới.

Chỉ dẫn PLAN (khung 5L): mỗi dòng ASSIGN | CREATE | EMPTY. Chọn id có sẵn, không clone. Gợi ý con của Origin, không auto-ATTACH cả nhánh. Đời Origin + chi kê lúc PLAN, chốt lúc phê.

UM: chỉ ADMIN, lô sau / cây lịch sử. Có đời. Sửa cùng hàng khi biết tên. USER lần 1 không UM.

Chọn trùng ông/bố giữa hai MFO: tốt — cùng member_id. Trùng = hai UUID một người thì mới chặn.

Duyệt V1: JWT CLAN_ADMIN/SYSTEM bấm máy. members.role Trưởng* = chức sở, map quyền duyệt = lát sau. Tự duyệt được, tự chịu.

Nợ / không lẫn PR

USER vẫn POST chi · cột max_generation_span chi vs trần lô 5 · unique UM · lịch sử members.role · 422 tự động từng ô · multi-home · 4 đời login.

Lát code đề xuất

Docs ticket PLAN|RESULT cửa PLAN khóa ATTACH trong khung RESULT picker FE siết POST chi / Founder=MWL.

Mở từng lát, hỏi rồi mới sinh mã. Q1 giữ máy Chi hiện tại.

5 sources

OK. Cần chú ý: bỏ quyền MWL tự do tạo branch và member.

Worked for 10s

Đã thống nhất để lập kế hoạch. File: artifacts/BFA-MFO-Doctrine-v1.2.0.md.

Đã live

Chi CRUD + cycle · enum/BPL/notify · Origin + quota cột · SUBMIT/APPROVE/REJECT (tem chi, PUT field mở, 409 khi duyệt lại) · Lát 1 founder/origin/ATTACH · smoke USER tạo chi + ATTACH + Admin ghi max_generation_span.

Đã chốt, chưa code hết

MFO = cửa sổ 5 dòng huyết thống, Origin = Dòng 0, ô trống được.

Lô ≠ chi. Mỗi lô sâu ≤ 5; ngang/sâu tiếp = lô mới + Origin cụ thể.

PLAN rồi RESULT. Chưa nghiệm thu (theo Founder+chi) thì không lô mới.

Chỉ dẫn = 5 ô ASSIGN | CREATE | EMPTY. Đời Origin + chi kê lúc PLAN, chốt lúc phê.

Origin lần 1 = member có sẵn; USER không UM. UM chỉ ADMIN.

Chọn, không clone. Gợi ý con Origin, không auto-ATTACH cả nhánh.

Founder = MWL. Trưởng* = members.role; bấm máy = JWT Admin. Tự duyệt = tự chịu.

MWL không tự do tạo branch và member — chỉ trong khung PLAN đã duyệt.

Nợ / không lẫn PR

USER vẫn POST chi · cột max_generation_span chi vs trần lô 5 · lịch sử members.role · unique UM · 422 tự động từng ô · multi-home · 4 đời login.

Lát code đề xuất

Docs ticket PLAN|RESULT cửa PLAN khóa ATTACH trong khung RESULT picker

FE siết POST chi / Founder=MWL.

Mở từng lát, hỏi rồi mới sinh mã. Q1 giữ máy Chi hiện tại.

OK.

Worked for 5s

Đã chốt bản tóm BFA-MFO-Doctrine-v1.2.0. Dùng làm SSOT khi lập lát.

Đã live: Chi CRUD · REVIEW API · Lát 1 gán · quota cột lúc Approve.

Chưa code: PLAN/RESULT lô 5 dòng · khóa ATTACH trong khung · cấm MWL tự do tạo branch/member · Founder = MWL · picker FE.

Lát code đầu tiên đề xuất: ticket PLAN|RESULT (payload 5 ô + Origin + đời/chi chốt lúc phê) — chưa form FE, chưa siết 422 từng ô.

Bảo «mở lát PLAN» thì mới sinh file. Q1 giữ máy Chi hiện tại.

mở lát PLAN

Worked for 16s

Đã thống nhất — dùng làm kế hoạch lát. File đủ header Q2:

Đã live — đừng làm lại

Chi CRUD + cycle · enum/BPL/notify · Origin + cột quota · SUBMIT/APPROVE/REJECT (tem chi, PUT field mở, 409 khi duyệt lại) · Lát 1 founder/origin/ATTACH · smoke: USER tạo chi + ATTACH, Admin ghi max_generation_span + Origin.

Đã chốt, chưa code hết

Đơn vị: Chi = tổ chức. MFO = cửa sổ 5 dòng huyết thống, Origin = Dòng 0. Lô = một lần xin-làm-tem. Nhiều lô / chi.

Hai trình: PLAN rồi RESULT. Chưa nghiệm thu (theo Founder+chi) thì không lô mới.

Chỉ dẫn PLAN = 5 ô: mỗi dòng ASSIGN | CREATE | EMPTY. Chọn id có sẵn, không clone. Gợi ý con Origin, không auto-ATTACH cả nhánh. Đời Origin + chi kê lúc PLAN, chốt lúc phê.

UM: chỉ ADMIN, lô sau / cây lịch sử. Có đời. Sửa cùng hàng khi biết tên. USER lần 1 không UM.

Chọn trùng ông/bố giữa hai MFO: tốt — cùng member_id. Trùng = hai UUID một người thì mới chặn.

Duyệt V1: JWT CLAN_ADMIN/SYSTEM bấm máy. members.role Trưởng* = chức sổ, map quyền duyệt = lát sau. Tự duyệt được, tự chịu.

Nợ / không lẫn PR

USER vẫn POST chi · cột max_generation_span chi vs trần lô 5 · lịch sử members.role · unique UM · 422 tự động từng ô · multi-home · 4 đời login.

Lát code đề xuất

Docs ticket PLAN|RESULT cửa PLAN khóa ATTACH trong khung RESULT picker
FE siết POST chi / Founder=MWL.

Mở từng lát, hỏi rồi mới sinh mã. Q1 giữ máy Chi hiện tại.

[BFA-MFO-Doctrine-v1.2.0.md](#) ở đâu nhỉ?

Worked for 18s

File nằm trong artifact phiên này (không tự lên GitHub CM-Docs). Tải bên dưới, copy vào CM-Docs/BFA/.

Đường dẫn máy: /home/workdir/artifacts/BFA-MFO-Doctrine-v1.2.0.md

Đường dẫn này ở đâu? /home/workdir/artifacts/BFA-MFO-Doctrine-v1.2.0.md

Tôi không biết. Chỉ dẫn cụ thể

Worked for 8s

/home/workdir/artifacts/ là thư mục trên máy Grok, không phải máy bạn. Không mở Finder tìm path đó.

Tải file ở thẻ dưới (nút download). Rồi:

Lưu vào máy, ví dụ ~/Downloads/BFA-MFO-Doctrine-v1.2.0.md

Copy vào repo docs: CM-Docs/BFA/BFA-MFO-Doctrine-v1.2.0.md

Commit + push CM-Docs

Đó là cách duy nhất lấy file — Grok không ghi trực tiếp vào GitHub hay ổ Mac.

[BFA-Branch-Family-Doctrine-v1.0.md](#)

Files

BFA — Học thuyết Chi / Gia đình

PATH : docs/BFA-Branch-Family-Doctrine-v1.0.md

DATETIME : 2026-09-12T10:45:00+07:00

VERSION : 1.0.0

STATUS : DRAFT-FOR-FREEZE (chờ Founder xác nhận)

SSOT-CÙNG : BFA-Branch-v1.1.0 · HandOver-M13-Branch-2026-09-11

KHÓA : Q1 bảo toàn · ALS · EDITOR ≠ users.status · place ≠ usage

A01 không sửa users.phone/email, members.gender, members.is_alive

Multi-home = nợ · RP/OP/SM CLOSED

Tài liệu này đóng ý nghĩa và phân tầng cửa. Không thay schema cho đến khi Lát được mở.

API M13 hiện tại (DRAFT / SUBMIT / APPROVE = tem) vẫn đúng với học thuyết này.

0. Vấn đề cần giải

Ba nhầm lẫn làm vỡ số:

Gộp chi (đơn vị tổ chức họ) với gia đình (hệ trục hệ của một người).

Gộp Tổ nghiệp (người đời $\leq n$, đã đứng sổ) với người khai chi (đời $n+1$, mở hồ sơ).

Gộp vai trên sổ (Founder / Origin) với tài khoản và quyền ghi (users.status, EDITOR).

Hậu quả đã thấy: gà-trứng founder / branch_id, clone người đời cũ, rác chi, khóa nhầm field khi APPROVED.

Học thuyết tách từng lớp, rồi mới nói cửa User / Admin.

1. Hai đơn vị — không đồng nghĩa

1.1 Chi / Ngành / Nhánh (branches)

Đơn vị tổ chức dòng họ. Một chi chứa nhiều hộ, nhiều đời, nhiều quan hệ — trong hạn ngạch đời Admin đã chốt.

Hiệu lực pháp lý trên sổ = tem APPROVED của CLAN_ADMIN / SYSTEM_ADMIN.

Hàng DRAFT / PROVISIONAL / SUBMITTED / REJECTED chưa là chi chính thức.

Cây chi (branches.parent_id) là cây tổ chức. Không thay cây người.

1.2 Gia đình tôi

Tập trục hệ của một member đã bind login: cha/mẹ tôi tôi vợ/chồng (nếu cùng họ, quyết riêng) con. Không mặc định gồm chú bác, anh em, ông bà như vai phải khai trên cửa User.

Gia đình tôi không sinh hàng branches. Nó sinh / sửa hàng members và father_id / mother_id.

1.3 Lý do tách

Nếu mỗi login = một chi, đời $n+2$ có hàng chục «chi» cùng một ông tổ — cây tổ chức không đọc được.

Nếu chi = hộ, toàn bộ parent_id chi, MERGE, offset đời, EDITOR chi, cấm USER tạo chi 2 trở thành thừa — trái BFA-Branch đã đóng.

Họ nhỏ, một hộ: hình trùng tạm thời. Mô hình vẫn hai lớp.

2. Ba vai — không gộp cột

2.1 Origin — Tổ nghiệp (origin_member_id, cột chưa có)

Người được suy tôn là tổ của chi.

Là hàng members đã có. Cấm clone.

Được phép đã có branch_id khác (đời $\leq n$ do Admin dựng sẵn).

Gắn Origin không đổi branch_id của người đó.

Có thể chưa đứng chi nào (branch_id null).

Lý do: người đời n là neo lịch sử, không phải nhân sự chi mới. Cướp hoặc nhân bản hàng member = dư thừa + xung đột.

2.2 Founder — Người tổ chức hồ sơ chi (founder_id, đã có)

Người đứng tên mở hồ sơ, hiểu sổ chi, chịu hạn ngạch đời.

Là hàng members. Không phải users.

Mang branch_id = chi này khi hồ sơ đã gắn (Lát 1).

Không mặc định = Origin.

Cửa User + grant: Founder thường = chính member login.

Cửa Admin: chọn tự do (chi lịch sử không cần login của tổ).

Lý do: «người khai» phải thuộc chi mới; «tổ» thường thuộc chi cũ. Một cột không gánh hai nghĩa.

2.3 EDITOR — Quyền ghi sổ chi

Cửa được phép tạo / lắp / (hạn chế) tháo member trong chi, trong quota.

Không đồng nhất users.status.

Không mặc định vĩnh viễn = Founder.

Founder mất hoặc không login: Admin giao EDITOR khác.

Founder có thể được giao EDITOR lúc APPROVE — đó là quyết định Admin, không phải hệ quả tự động của cột founder_id.

Lý do: khóa đã đóng «EDITOR chi ≠ users.status». Gộp Founder = full editor khóa sổ khi người khai không còn dùng được tài khoản.

2.4 User login

Tài khoản. Chỉ có nghĩa khi đã bind users members (hạng mục 4 đời login — chưa xong thì cửa «Gia đình tôi» / «xin Founder» chưa mở cho User).

User không phải Origin, không phải Chi.

3. Máy trạng thái chi (đã đóng)

DRAFT | PROVISIONAL submit SUBMITTED approve APPROVED
MERGED (lát sau)

reject REJECTED submit lại (ticket mới; ticket cũ WITHDRAWN)
Tạo DRAFT / PROVISIONAL: rỏ, chưa hiện sổ chính thức.

APPROVED = tem thừa nhận. Không niêm phong tên, mô tả, origin, parent_id (khóa field là lát riêng nếu cần).

APPROVED không reject / không submit lại.

REJECTED được submit lại: ticket BRANCH_REVIEW mới; ticket cũ WITHDRAWN.

MERGED = gán cây tổ chức (parent_id + offset đời) — chưa mở.

Lý do: chống rác ở cửa duyệt, không ở cấm nháp. Nháp xóa/ARCHIVED được; tem không đảo.

Mặc định schema CRUD hiện tại: chi mới = PROVISIONAL. Submit chấp nhận DRAFT | REJECTED | PROVISIONAL. Không mở lại OP/SM.

4. Điều kiện từng cửa

4.1 POST tạo hàng chi (nháp)

CLAN_ADMIN / SYSTEM_ADMIN: luôn được.

USER: không mặc định được. Chỉ khi có grant Founder (ticket / cờ — chưa làm).

Không đòi Origin, không đòi 4 đời, không đòi cặp huyết thống.

Lý do: cấm POST trước khi có chi = gà-trứng «Founder của chi nào?». Chống rác bằng Submit + tem, không bằng cấm nháp.

Route hiện USER được POST /branches — lệch mục 4.1. Nợ siết.

4.2 Submit (đã cài)

Bắt buộc:

Người gọi: chủ hồ sơ (changed_by) nếu USER; hoặc CLAN_ADMIN / SYSTEM.

Status: DRAFT | REJECTED | PROVISIONAL.

Có founder_id.

Có ≥ 1 cặp cha/mẹ-con cùng branch_id chi này.

422 BRANCH_SUBMIT_PRECOND nếu thiếu, kèm missing.

Nợ khi có cột Origin: thêm Origin vào precondition Submit.

Không đưa «đủ 4 đời trên sổ» vào precondition Submit — đưa vào quota sau tem.

Lý do: 4 đời là hạn ngạch hiểu biết được Admin chốt, không phải điều kiện tồn tại hồ sơ.
Số thật thường thiếu ông cố.

4.3 Approve

Chỉ CLAN_ADMIN / SYSTEM_ADMIN.

Trên ticket BRANCH_REVIEW Admin ghi:

Tem APPROVED.

Quota đời (max_generation_span) — số đời được phép tính từ Origin (khi đã có cột) hoặc từ Founder (tạm).

Có thể giao EDITOR.

Nới quota sau này = ticket mới. Không sửa ý nghĩa tem cũ.

4.4 Reject

reason bắt buộc. Chi REJECTED. Ticket REJECTED. Gửi lại = ticket mới.

5. Quota đời

Sau APPROVE, Chi được phép sâu tối đa N đời.

Mốc: đời Origin = neo (0 hoặc members.generation của Origin trên sổ toàn họ).

Span trên chi = max(đời người đã gắn chi này) – đời Origin.

Vượt quota: 422 lúc gắn / tạo member vào chi, không lúc sửa tên chi.

Đếm chỉ trên father_id / mother_id đã ghi + branch_id chi này. Không đếm chú bác nếu chưa có trên cây người.

Lý do: Admin biết độ tin cậy của Founder và độ dày sổ họ. Founder bị giới hạn bởi sự hiểu biết đã được tem, không phải bởi ham ghi.

Cột max_generation_span chưa có — nợ schema.

6. Cửa User — Gia đình tôi

Áp khi user đã bind member.

Được:

Tạo / sửa người trực hệ: cha/mẹ (nếu chưa có trên sổ), mình (đã có), con.

Ghi father_id / mother_id (kể cả khi mình sẽ là Founder).

Origin mặc định gợi ý = father_id của mình — không bắt nếu thiếu.

Không được mặc định:

POST chi.

Khai chú bác, anh em như thành viên bắt buộc của «Gia đình tôi».

Đổi branch_id người đời $\leq n$ đã đứng chi khác.

Tạo chi thứ hai (nợ đã ghi).

Thiếu cha trên sổ: vẫn khai mình + con; Submit chi (nếu có grant) 422 vì thiếu Origin / thiếu cặp — đúng.

Hai anh em cùng bố: hai gia đình tôi, cùng Origin, một chi do Admin / Founder lắp — không hai chi.

7. Cửa Admin

Dựng chi lịch sử tới đời n mà không cần login của tổ.

Chọn Origin = người đời $\leq n$; không đổi branch_id người đó.

Phong / thu Founder và EDITOR.

Lắp hộ (gia đình) vào chi: ghi members.branch_id trong quota.

Tem APPROVE / REJECT + quota.

Ghép cây chi (lát MERGED).

Admin không bị luật «cha của người login = Origin».

8. Gà-trứng — thứ tự bắt buộc

Chi và người sinh độc lập. Gặp nhau ở bước gán. Khóa ở bước gửi.

9. Nhập người (CRUD members) và/hoặc tạo chi nháp

10. Gắn tổ chức: Origin (trở, không chuyển chi)

Founder + members.branch_id (người của chi mới)

11. Cặp huyết thống cùng branch_id (thường Founder + con)

12. Submit Admin tem + quota

Không tạo member mà lúc POST chi.

Không tạo chi ẩn lúc POST member.

Không nối FK founder_id sang users.

Khi tạo member từ màn chi: luôn cho nhập cha/mẹ, kể cả Founder. Không bắt cha/mẹ trên chi này khi tạo Founder.

9. Tháo / chuyển người

Trạng thái chi

Founder / EDITOR tháo branch_id

DRAFT, PROVISIONAL, REJECTED

Được, trong tenant

SUBMITTED, UNDER_REVIEW

Không — đang xét

APPROVED, MERGED

Không tự do. Proposal hoặc Admin

Lý do: không đảo tem, cũng không để rút nửa số đã công nhận.

10. Chọn người có sẵn

Gợi ý, không chặn cứng đồng danh:

Tên, alias, đời, sống/mất, cha/mẹ nếu có, chi hiện tại (có thể chưa có / unknown).

Cảnh báo nếu người đã đứng chi khác — Origin thì cho trở; Founder / thành viên chi mới thì không cướp branch_id.

11. Note và VoiceHelp

members.note, branches.description là định hướng cho người sau (vì sao tách chi, nguồn gia phả, chỗ không chắc).

VoiceHelp dẫn cách viết Note — tài liệu UX, không thêm cột. Không biến Note thành field bắt buộc Submit.

12. Khóa và nợ — không làm chệch doctrine

Đã khóa:

ALS; không nhét tenant_id vào findUnique / update.

A01: không sửa users.phone, users.email, members.gender, members.is_alive.

EDITOR ≠ users.status.

place ≠ usage (member_residences).

Không sửa schema audit_logs.

members.phone / email không unique.

Multi-home (một người hai họ / hai chi hành chính) = nợ.

RP / OP / SM không mở lại.

Nợ schema / lát (chưa sinh):

origin_member_id

max_generation_span

Grant Founder trước khi USER được POST chi

Siết route POST /branches bỏ USER mặc định

Bind 4 đời login

EDITOR grant

MERGED + offset đời

Cấm USER tạo chi 2

Form FE duyệt / VoiceHelp

API đã có: catalog silentIntent, SUBMIT / APPROVE / REJECT, tem không khóa field, ticket mới khi gửi lại.

13. Một trang — câu đóng

Chi là tổ chức họ, do Admin dán tem.

Gia đình tôi là trực hệ của member login, không phải chi.

Origin là tổ nghiệp, không chuyển chi.

Founder là người thực hiện tổ chức cây trực hệ (bắt buộc phải có tài khoản login) bắt đầu từ gốc (là Origin). Cây trực hệ, mà Founder là thành phần trực tiếp nó, có thể không thuộc chi nào. Khi duyệt, ADMIN có thể thực hiện gán Chi cho cây trực hệ. mà founder đã trình.

EDITOR là quyền ghi, giao riêng.

Nháp thì rẻ. Tem thì một chiều. Quota đời do Admin chốt trên ticket.

Người và chi sinh độc lập; gán rồi mới gửi; gửi rồi mới thành chi.

Hết.

Tiếp Module Chi Branches - Grok